



Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Inteligencia Artificial

Predicción de Cadenas de Ataque en Infraestructuras Kubernetes mediante Graph Neural Networks

Trabajo Fin de Estudios

presentado por: Milton Obed Rayo Viana

Dirigido por: Pedro León Millán

Ciudad: Medellín

Fecha: Mayo de 2026

Índice de Contenidos

Resumen	VII
Abstract	VIII
1 Introducción	1
1.1 Motivación	1
1.2 Planteamiento del problema	2
1.3 Objetivos generales	2
1.4 Estructura del trabajo	2
2 Estado del Arte	4
2.1 Seguridad en infraestructuras Kubernetes	4
2.2 Análisis estático de IaC	5
2.3 Aprendizaje automático para detección de vulnerabilidades	6
2.4 Redes neuronales de grafos	9
2.5 GNNs aplicadas a ciberseguridad	10
2.6 Detección de cadenas de ataque y grafos de ataque en Kubernetes	11
2.7 Síntesis y posicionamiento	12
3 Objetivos y Metodología	14
3.1 Objetivo general	14
3.2 Objetivos específicos	14
3.3 Metodología	15
3.4 Estrategia de partición y validación de datos	16
3.5 Métricas de evaluación	17
4 Diseño e Implementación	18
4.1 Identificación de requisitos	18

4.1.1	Requisitos funcionales	18
4.1.2	Requisitos no funcionales	19
4.2	Arquitectura general del sistema	19
4.3	Construcción del dataset	20
4.3.1	Fuentes de datos	20
4.3.2	Características extraídas	21
4.4	Construcción del grafo de clúster	22
4.4.1	Detección de escape y movimiento lateral	23
4.4.2	Detección de privilegio RBAC	24
4.5	Capa 1: Random Forest	24
4.5.1	Modelo binario	24
4.5.2	Modelo de severidad	25
4.6	Capa 2: Graph Attention Network	25
4.6.1	Arquitectura	25
4.6.2	Entrenamiento	26
4.6.3	Aumentación de datos	27
4.7	Capa 3: Ensemble con Algoritmo Genético	27
4.8	Implementación de kubescan	28
4.9	Tabla de características del sistema	29
4.10	Calidad de software y proceso de validación	29
5	Resultados y Evaluación	33
5.1	Resultados de la Capa 1: Random Forest	33
5.1.1	Clasificación binaria	33
5.1.2	Importancia de características	34
5.1.3	Modelo de severidad	35
5.2	Resultados de la Capa 2: GNN	36
5.2.1	Evolución por fase experimental	36
5.2.2	Análisis de Precisión@5	37
5.2.3	Resultados por pliegue (fase final)	37
5.3	Resultados de la Capa 3: Ensemble	39
5.4	Análisis de la clasificación por clases: GNN	40
5.5	Validación con herramientas de análisis estático	41

5.6	Evaluación funcional de la herramienta	42
5.7	Comparación con el estado del arte	42
5.7.1	Capa 1 en contexto	43
5.7.2	Capa 2 en contexto	43
6	Conclusiones y Trabajo Futuro	45
6.1	Conclusiones	45
6.2	Consideraciones éticas y uso responsable	46
6.3	Limitaciones	47
6.4	Trabajo futuro	49
A	Acrónimos y Abreviaturas	54
B	Repositorio de Código y Datos	56
C	Guía de Uso y Reproducción	57
C.1	Instalación	57
C.2	Uso de la interfaz de línea de comandos	57
C.2.1	Análisis de un directorio de manifiestos	57
C.2.2	Análisis de un clúster en ejecución	60
C.3	Reproducción del pipeline de investigación	60

Índice de Ilustraciones

4.1	Arquitectura del sistema <i>kubescan</i> : pipeline de tres capas desde manifiestos YAML Ain't Markup Language (YAML) de entrada hasta la clasificación de riesgo del clúster. Los <i>risk_scores</i> producidos por la Capa 1 se incorporan como atributo nodal 26 del grafo de entrada de la Capa 2.	20
4.2	Ejemplo de grafo de clúster Kubernetes con cuatro nodos, mostrando los tres tipos de nodo (escape, movimiento lateral y limpio) y las aristas dirigidas que cada uno origina.	23
5.1	Matriz de confusión del modelo Random Forest en el conjunto de test (496 muestras). Los 3 falsos positivos son manifiestos seguros con alta densidad de características de riesgo; no existe ningún falso negativo.	34
5.2	Importancia de las 10 características más relevantes del clasificador Random Forest. Las dos primeras (en rojo) concentran el 48 % de la capacidad discriminativa total del modelo.	35
5.3	Evolución de F1 macro y Precisión@5 de la Capa 2 a lo largo de las cuatro primeras fases experimentales (protocolo preliminar).	38
C.1	Flujo de ejecución de <i>kubescan scan</i> sobre el ejemplo <i>produccion-web</i> , desde la invocación por Command-Line Interface (Interfaz de Línea de Comandos) (CLI) hasta el veredicto final.	58

Índice de Tablas

2.1	Comparativa de algoritmos de ML para clasificación de manifiestos Infrastructure as Code (Infraestructura como Código) (IaC)	8
2.2	Comparativa de herramientas de seguridad para Kubernetes	12
4.1	Composición del conjunto de datos de manifiestos Kubernetes	21
4.2	Tipos de aristas en el grafo de clúster	22
4.3	Características del clasificador Random Forest (RF) — grupo Rahman (1/2)	30
4.4	Características del clasificador RF — grupos derivado y extendido, y nodo (2/2)	31
5.1	Resultados del Random Forest — clasificación binaria	33
5.2	F1 por clase — modelo de severidad (3 clases, test)	36
5.3	Evolución de resultados de la Capa 2 (Graph Attention Network (Red de Atención sobre Grafos) (GAT), 5-fold Cross-Validation (Validación Cruza- da) (CV)). Las fases 1–4 emplean el protocolo de particionado preliminar (con fuga por variantes aumentadas) y se reportan como registro histórico; la fase 5 es el resultado final con protocolo <i>group-aware</i>	37
5.4	Resultados por pliegue — Capa 2 final (protocolo <i>group-aware</i> ; los pliegues particionan 81 grafos originales, 21 cadenas; test excluido)	38
5.5	Ablación de componentes del ensemble en el conjunto de test (15 grafos, 4 cadenas, techo $P@5 = 0,80$)	40
5.6	Ablación de arquitectura — Capa 2 (5-fold CV, protocolo <i>group-aware</i>) . .	41

Índice de Ecuaciones

2.1	Actualización de la representación de un nodo en una capa GNN	9
2.2	Coefficiente de atención en Graph Attention Networks	9
4.1	Función de puntuación del ensemble (Capa 3)	20
4.2	Cálculo de pesos de clase para el entrenamiento de la GAT	26
4.3	Función objetivo (fitness) del Algoritmo Genético	28

Resumen

Este trabajo presenta *kubescan*, un sistema de detección automática de cadenas de ataque en infraestructuras *Kubernetes* que combina un clasificador *Random Forest* (Capa 1) con una Red Neuronal de Grafos de Atención — *Graph Attention Network* (GAT) — (Capa 2) y optimización de pesos mediante Algoritmo Genético (Capa 3).

A partir de 2.479 manifiestos de *Kubernetes* procedentes de cuatro fuentes públicas complementarias, se construyen 96 grafos de clúster con aristas que representan relaciones de privilegio, movimiento lateral y proximidad. La Capa 1 alcanza una F1 macro de 0,9935 en la clasificación de manifiestos individuales, con $AUC = 0,9980$ y cero falsos negativos en el conjunto de test. La Capa 2, basada en GAT de 3 capas con 4 cabezas de atención, *embedding* de tipos de arista y aumentación de grafos, logra $F1 \text{ macro} = 0,870 \pm 0,075$ y $\text{Precisión}@5 = 0,720 \pm 0,098$ en validación cruzada con un protocolo de particionado consciente de grupos (sin fuga entre variantes aumentadas y sus grafos base), superando el objetivo de diseño de $P@5 \geq 0,70$ sobre un corpus de 96 grafos de clúster originales (471 tras aumentación de datos). En el conjunto de test — excluido de la validación cruzada y del ajuste del *ensemble* — el sistema alcanza $\text{Precisión}@1 = 1,00$, recupera 3 de las 4 cadenas de ataque en las primeras 5 posiciones y no produce falsos positivos de clústeres limpios ($FPR_{\text{clean}} = 0 \%$). Los resultados, acompañados de intervalos de confianza acordes al tamaño del corpus, son consistentes con la viabilidad del enfoque híbrido GNN + RF para la detección automática de rutas de explotación en entornos *Kubernetes* reales.

Palabras clave: Kubernetes, Graph Neural Networks, Random Forest, seguridad, cadenas de ataque, infraestructura como código, misconfiguraciones.

Abstract

This work presents *kubescan*, an automatic attack-chain detection system for *Kubernetes* infrastructures that combines a Random Forest classifier (Layer 1) with a Graph Attention Network (GAT) (Layer 2) and Genetic Algorithm ensemble weight optimisation (Layer 3).

From 2,479 *Kubernetes* manifests drawn from four complementary public sources, 96 cluster graphs are built with edges representing privilege-escalation, lateral-movement, and proximity relationships. Layer 1 achieves $\text{macro-F1} = 0.9935$, Area Under the Curve (Área Bajo la Curva ROC) ($\text{AUC} = 0.9980$), and zero false negatives on the test set. Layer 2, a 3-layer GAT with 4 attention heads, edge-type embeddings and graph augmentation, achieves $\text{macro-F1} = 0.870 \pm 0.075$ and $\text{Precision@5} = 0.720 \pm 0.098$ under a group-aware cross-validation protocol (no leakage between augmented variants and their base clusters), exceeding the $\text{P@5} \geq 0.70$ design target for cluster-level attack-chain classification. On the test set — held out from cross-validation and ensemble tuning — the system achieves $\text{Precision@1} = 1.00$, recovers 3 of the 4 attack chains within the top-5 ranking, and produces no false positives on clean clusters ($\text{FPR}_{\text{clean}} = 0$). Results, reported with confidence intervals appropriate to the corpus size, are consistent with the feasibility of the hybrid Graph Neural Network (Red Neuronal de Grafos) (GNN) + RF approach for automatic exploitation-path detection in real *Kubernetes* environments.

Keywords: *Kubernetes*, Graph Neural Networks, Random Forest, security, attack chains, infrastructure as code, misconfigurations.

1. Introducción

1.1. Motivación

La adopción masiva de *Kubernetes* como plataforma de orquestación de contenedores ha transformado la manera en que las organizaciones despliegan y gestionan sus aplicaciones. La *Cloud Native Computing Foundation* reporta que el 96 % de las organizaciones utilizan o evalúan *Kubernetes*, y que el 53 % ha sufrido un incidente de seguridad en entornos de contenedores en el último año (Cloud Native Computing Foundation, 2022). Sin embargo, esta proliferación ha ampliado considerablemente la superficie de ataque: una sola misconfiguration en un manifiesto YAML puede habilitar la escalada de privilegios desde un contenedor comprometido hasta el nodo anfitrión del clúster o, en casos extremos, hasta la infraestructura subyacente completa (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022).

Los enfoques de análisis estático tradicionales —herramientas como *Checkov* (Bridgecrew, 2021) o *kube-linter* (StackRox, 2021)— detectan misconfiguraciones aisladas en manifiestos individuales, pero no modelan las relaciones entre recursos del clúster. Una única flag de privilegio en un pod no es suficiente para determinar si existe una cadena de ataque viable; es la combinación de nodos de escape con recursos de movimiento lateral la que habilita rutas de explotación completas, tal como describen las técnicas de escalada de privilegios en contenedores documentadas por Hutchins, Cloppert, y Amin 2011 e incorporadas en el marco MITRE ATT&CK for Containers (MITRE Corporation y Center for Threat-Informed Defense, 2021).

Herramientas más recientes como *KubeHound* (Datadog Security Labs, 2023) e *IceKube* (WithSecure Labs, 2023) abordan este problema construyendo grafos de ataque sobre clústeres activos. Sin embargo, requieren acceso con privilegios al servidor API de *Kubernetes*, lo que limita su aplicabilidad como control preventivo en el pipeline de integración y entrega continua (CI/CD) —un principio conocido como *shift-left security*: adelantar los controles de seguridad a las etapas más tempranas del desarrollo.

Este trabajo aborda ese vacío: construir un sistema capaz de razonar sobre la *estructura* del clúster a partir de manifiestos YAML estáticos, sin requerir un clúster activo, para identificar automáticamente cadenas de ataque potenciales antes de que sean desplegadas.

1.2. Planteamiento del problema

Formalmente, dado un conjunto de manifiestos YAML que describen un clúster *Kubernetes*, el problema consiste en:

1. Clasificar cada manifiesto individual como seguro o misconfigured (Capa 1).
2. Modelar el clúster como un grafo heterogéneo y clasificarlo en una de tres categorías: *clean* (sin misconfiguraciones relevantes), *isolated* (misconfiguraciones sin cadena viable) o *attack_chain* (existe una ruta de escalada de privilegios hasta movimiento lateral) (Capa 2).
3. Combinar las señales de ambas capas mediante una función de puntuación optimizada para maximizar la Precisión@5 —la fracción de clústeres de cadena de ataque en el top-5 del ranking— minimizando falsos positivos (Capa 3).

La restricción de operar sobre manifiestos YAML estáticos —sin requerir acceso al clúster activo— es un requisito de diseño explícito que diferencia el presente trabajo de las soluciones de grafo de ataque existentes y lo posiciona como un control de seguridad integrable en pipelines CI/CD.

1.3. Objetivos generales

El objetivo general es diseñar, implementar y evaluar un sistema de detección de cadenas de ataque en *Kubernetes* que complemente los enfoques de análisis estático por manifiesto individual con el contexto estructural del clúster, mediante la combinación de aprendizaje automático clásico y aprendizaje sobre grafos, operando principalmente sobre manifiestos YAML estáticos, si bien incorpora opcionalmente una vía de consulta directa a un clúster activo (Capítulo 3).

1.4. Estructura del trabajo

El trabajo se organiza como sigue. El Capítulo 2 revisa el estado del arte en seguridad de *Kubernetes*, herramientas de análisis estático y grafos de ataque, redes neuronales de grafos y detección de cadenas de ataque; incluye una tabla comparativa de las herramientas más relevantes y un análisis del posicionamiento de *kubescan*. El Capítulo 3 define los

objetivos específicos y la metodología de cinco fases. El Capítulo 4 describe el diseño e implementación del sistema, incluyendo el análisis de requisitos, la arquitectura de tres capas, la construcción del dataset y el grafo, y la tabla completa de las 25 características del clasificador. El Capítulo 5 presenta los resultados experimentales de cada capa, la validación cruzada con herramientas externas y la evaluación funcional de la herramienta. El Capítulo 6 recoge las conclusiones vinculadas a cada objetivo, las limitaciones, las consideraciones éticas y las líneas de trabajo futuro.

2. Estado del Arte

La detección automática de vulnerabilidades en infraestructuras *Kubernetes* se encuentra en la intersección de cuatro áreas de investigación activa: el análisis de seguridad en infraestructura como código (*Infrastructure as Code*, IaC), el aprendizaje automático aplicado a la ciberseguridad, las redes neuronales de grafos para razonamiento estructural, y la construcción de grafos de ataque para la identificación de rutas de explotación. A continuación se revisan los trabajos y herramientas más relevantes en cada área, y se posiciona la contribución de este trabajo respecto a ellos.

2.1. Seguridad en infraestructuras Kubernetes

Kubernetes es la plataforma de orquestación de contenedores dominante en entornos de producción empresariales, con un 96 % de organizaciones que la utilizan o evalúan según la encuesta anual de la *Cloud Native Computing Foundation* (Cloud Native Computing Foundation, 2022). A pesar de su adopción generalizada, la seguridad sigue siendo uno de los principales retos operacionales: el informe *State of Kubernetes Security* de Red Hat 2024 (Red Hat, 2024) revela que el 89 % de las organizaciones sufrió al menos un incidente de seguridad relacionado con *Kubernetes* en el último año, y que el 67 % retrasó despliegues a producción por preocupaciones de seguridad. El 46 % reportó pérdidas de clientes o ingresos directamente atribuibles a incidentes en entornos de contenedores. Entre los riesgos más temidos, las vulnerabilidades representan el 33 % de las preocupaciones de los equipos, mientras que las misconfiguraciones y exposiciones alcanzan el 27 %.

Su modelo declarativo, basado en ficheros YAML que especifican el estado deseado del clúster, ofrece grandes ventajas en reproducibilidad y automatización, pero también introduce una amplia superficie de ataque: un fichero mal configurado desplegado en producción puede conceder privilegios excesivos a un contenedor o exponer secretos en texto plano. La flexibilidad de *Kubernetes* —que permite configurar desde el sistema de ficheros del anfitrión hasta los permisos de red a nivel de pod— implica que el número de vectores de ataque crece proporcionalmente con la riqueza del modelo de configuración.

La Agencia de Seguridad Nacional de Estados Unidos (NSA) y la Agencia de Ciberseguridad e Infraestructura (CISA) publicaron en 2022 una guía técnica (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022) que enumera las mis-

configuraciones más críticas: uso de `privileged: true`, montaje del socket de Docker, compartición del espacio de nombres de proceso del anfitrión (*hostPID*), y automontaje de tokens de cuenta de servicio con permisos excesivos. Estos vectores son precisamente los que este trabajo modeliza como nodos de escape en el grafo del clúster.

Los trabajos de BishopFox (Art, 2021) demostraron empíricamente que un único pod con configuración *everything-allowed* puede comprometer el nodo anfitrión en segundos y pivotar lateralmente a través del clúster. La plataforma educativa *Kubernetes Goat* (Madhuakula, 2020) sistematiza 14 escenarios de vulnerabilidad con los que se pueden reproducir rutas de explotación completas en un entorno controlado. Ambas fuentes son la base de las etiquetas de cadena de ataque del presente trabajo.

El marco taxonómico de referencia para los ataques en entornos de contenedores y *Kubernetes* es *MITRE ATT&CK for Containers* (MITRE Corporation y Center for Threat-Informed Defense, 2021), incorporado en la versión 9 de la base de conocimiento ATT&CK. Este marco describe las tácticas y técnicas empleadas por adversarios a nivel de orquestador (*Kubernetes*) y a nivel de contenedor (*Docker*), incluyendo las que este trabajo detecta directamente: escape de contenedor hacia el nodo anfitrión (T1611), movimiento lateral mediante abuso de cuentas de servicio con permisos excesivos (T1078.004, *Valid Accounts: Cloud Accounts*), y escalada de privilegios mediante permisos RBAC de escalada (T1548, *Abuse Elevation Control Mechanism*). La alineación entre las *escape flags* del presente sistema y las técnicas ATT&CK valida que el espacio de detección cubre los vectores de ataque documentados y más explotados en la industria.

2.2. Análisis estático de IaC

El análisis estático de manifiestos IaC cuenta con una larga trayectoria en la literatura. Rahman et al. (Rahman, Shamim, Bose, y Pandita, 2023) construyeron el primer conjunto de datos de referencia para manifiestos *Kubernetes* extraídos de repositorios públicos de GitHub y GitLab, etiquetados mediante análisis estático multi-herramienta. Su conjunto de datos —la base del presente trabajo— contiene más de 1.900 manifiestos con 57 columnas de características binarias que codifican la presencia o ausencia de patrones de *misconfiguration*.

Las herramientas industriales actuales más utilizadas son *Checkov* (Bridgecrew, 2021), que verifica más de 400 políticas de seguridad predefinidas, y *kube-linter* (StackRox, 2021),

especializado en buenas prácticas de *Kubernetes*. Ambas operan al nivel de manifiesto individual y no detectan patrones distribuidos entre varios recursos del clúster. También destaca *Kubescape* (ARMO Ltd., 2021), una plataforma de seguridad de código abierto integrada en la *Cloud Native Computing Foundation* que combina análisis estático con validación de cumplimiento contra los marcos National Security Agency (NSA), MITRE ATT&CK y CIS *Benchmarks*. Al igual que Checkov y kube-linter, sin embargo, Kubescape evalúa recursos de forma independiente y no construye ningún modelo del clúster como estructura relacional.

La noción de *Unified Misconfiguration Index* (UMI), propuesta por Malul, Meidan, Mimran, Elovici, y Shabtai 2024 en *GenKubeSec*, agrega las señales de múltiples herramientas en un único índice de riesgo, conceptualmente equivalente al *risk_score* generado por la Capa 1 del presente sistema. Su propuesta de combinar *Checkov*, *kube-linter* y un modelo de lenguaje de gran escala (LLM) es complementaria a la del presente trabajo, aunque opera en el mismo nivel de manifiesto individual.

Una limitación estructural de todas las herramientas anteriores es que operan sobre manifiestos en aislamiento: saben que un pod tiene `privileged: true`, pero no saben si ese pod tiene acceso de red a los recursos críticos del clúster. La detección de cadenas de ataque requiere, inevitablemente, razonar sobre la estructura del clúster como un todo.

2.3. Aprendizaje automático para detección de vulnerabilidades

El aprendizaje automático se ha aplicado a la detección de vulnerabilidades en código fuente y configuraciones desde hace más de una década. Los clasificadores basados en *Random Forest* (Breiman, 2001) han demostrado resultados robustos en escenarios de alta dimensionalidad y clases desbalanceadas —características típicas de los conjuntos de datos de seguridad— gracias a su capacidad de combinar árboles de decisión con ponderación de clases y selección aleatoria de características (*max_features=sqrt*). El proceso de selección de hiperparámetros para *Random Forest* en tareas de seguridad sigue generalmente la estrategia de búsqueda en rejilla (*grid search*) con validación cruzada estratificada, explorando el espacio de `n_estimators` (100–1.000), `max_depth` (sin límite vs. 10–20) y `min_samples_leaf` (1–5), y seleccionando la combinación que maximiza la métrica objetivo sobre los pliegues de validación (Pedregosa y cols., 2011).

En el dominio de la detección de intrusiones en red, el conjunto de datos UNSW-NB15 (Moustafa y Slay, 2015) se ha convertido en un referente de evaluación para clasificadores supervisados, y presenta un desequilibrio de clases del mismo orden de magnitud que el conjunto de manifiestos de este trabajo (64,2 % seguros vs. 35,8 % misconfigured). El desequilibrio de clases es un problema generalizado en los datasets de seguridad: las misconfiguraciones críticas son rarísimas en repositorios reales, lo que obliga a adoptar estrategias explícitas de compensación. La alternativa más común es asignar pesos de clase inversamente proporcionales a su frecuencia (*class_weight=balanced* en *scikit-learn*), que es la estrategia adoptada en este trabajo (Pedregosa y cols., 2011). Otras alternativas como SMOTE (*Synthetic Minority Over-sampling Technique*) no se consideraron apropiadas dado que las características son binarias, y la interpolación lineal entre muestras binarias puede generar vectores fuera del espacio semántico válido.

Trabajos más recientes han explorado el uso de modelos de lenguaje de gran escala (Large Language Model (Modelo de Lenguaje de Gran Escala) (LLM)s) para la detección y remediación de misconfiguraciones en *Kubernetes* (Malul y cols., 2024). Si bien los LLMs ofrecen capacidades de razonamiento semántico superiores, requieren recursos computacionales sustancialmente mayores, producen salidas no deterministas y son más opacos que los clasificadores clásicos, lo que dificulta su interpretación en auditorías de seguridad donde la trazabilidad de las reglas de detección es un requisito normativo. Para entornos de producción donde el operador de seguridad debe poder justificar ante una auditoría por qué un clúster fue marcado como de alto riesgo, la interpretabilidad del *Random Forest* —mediante importancia de características y reglas de árbol legibles— representa una ventaja operacional significativa sobre los modelos de caja negra.

La elección de *Random Forest* sobre otros algoritmos clásicos para la clasificación de manifiestos IaC responde a tres criterios documentados en la literatura de aprendizaje automático aplicado a la seguridad:

1. **Robustez ante datos binarios y escasos.** Las características del presente trabajo son en su mayoría binarias (presencia/ausencia de una misconfiguration), con alta proporción de ceros. Los *Random Forests* manejan este tipo de datos de forma nativa sin requerir normalización, a diferencia de las Máquinas de Vectores Soporte (SVM) o las Redes Neuronales que son sensibles a la escala de las características (Breiman, 2001).

2. **Gestión intrínseca del desequilibrio de clases.** El parámetro `class_weight=balanced` de *scikit-learn* implementa una estrategia de ponderación inversamente proporcional a la frecuencia de clase, lo que eleva efectivamente el coste de clasificar erróneamente las muestras de la clase minoritaria (misconfiguraciones críticas) sin modificar la función de pérdida subyacente (Pedregosa y cols., 2011).
3. **Generación de probabilidades calibradas.** El promedio de los votos del *ensemble* de árboles produce probabilidades suaves en $[0, 1]$ que pueden utilizarse directamente como características continuas (*risk_scores*) en el grafo de la Capa 2. Algoritmos como los Árboles de Decisión individuales o los *k*-Nearest Neighbours producen probabilidades menos calibradas en presencia de desequilibrio de clases.

Tabla 2.1: Comparativa de algoritmos de ML para clasificación de manifiestos IaC

Algoritmo	Binario	Desequilibrio	Prob. cali- brada	Observación
Random Forest	✓	✓	Buena	Adoptado (Capa 1)
Árbol de Decisión	✓	Parcial	Pobre	Alta varianza en grafos
Support Vector Machine (Máquina de Vectores Soporte) (SVM) (RBF)	✓	Parcial	Requiere Platt	Entrenamiento $O(n^2)$ – $O(n^3)$; viable pero menos eficiente
Red Neuronal (MLP)	✓	Con pesos	Variable	Requiere normalización
<i>k</i> -NN	✓	No	Estimada	Costoso en inferencia

Fuente: elaboración propia.

Trabajos como Wist, Hammer, y Yazidi 2021, que aplican técnicas de Machine Learning (Aprendizaje Automático) (ML) a la predicción de vulnerabilidades en código fuente, confirman que los *Random Forests* ofrecen sistemáticamente mejor rendimiento que los Árboles de Decisión individuales y comparable al de las Redes Neuronales superficiales en espacios de características binarias de alta dimensión. Esta evidencia, combinada con las ventajas de interpretabilidad para auditorías de seguridad, motivó la adopción del *Random Forest* como componente de Capa 1 de *kubescan*.

2.4. Redes neuronales de grafos

Las redes neuronales de grafos (GNNs) han emergido como la herramienta principal para aprendizaje en datos con estructura relacional (Hamilton, 2020; Zhou y cols., 2020). A diferencia de los clasificadores tabulares clásicos, las GNNs operan directamente sobre la topología del grafo, propagando información entre vecinos mediante operaciones de paso de mensajes (*message passing*). Formalmente, en una capa GNN la representación del nodo v se actualiza como:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{AGGREGATE}\left(\left\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\right\}\right) + \mathbf{b}^{(l)}\right) \quad (2.1)$$

donde $\mathcal{N}(v)$ es el conjunto de vecinos de v , $\mathbf{W}^{(l)}$ y $\mathbf{b}^{(l)}$ son parámetros entrenables de la capa l , y σ es una función de activación no lineal. La elección de la función AGGREGATE diferencia las principales arquitecturas.

Las arquitecturas más relevantes para este trabajo son:

Graph Convolutional Networks (GCN) (Kipf y Welling, 2017): implementan AGGREGATE como la media ponderada de los estados de los vecinos normalizada por sus grados. Son eficientes pero asignan pesos uniformes a todos los vecinos, sin distinción entre tipos de arista.

GraphSAGE (Hamilton, Ying, y Leskovec, 2017): extensión inductiva que permite aplicar el modelo a grafos no vistos durante el entrenamiento, relevante en escenarios donde aparecen nuevos clústeres en producción. La inducción se logra aprendiendo funciones de agregación en lugar de *embeddings* fijos.

Graph Attention Networks (GAT) (Veličković y cols., 2018): sustituyen la normalización fija de Graph Convolutional Network (Red Convolutiva de Grafos) (GCN) por coeficientes de atención aprendidos. Para cada par de nodos (v, u) el coeficiente de atención es:

$$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]))}{\sum_{k \in \mathcal{N}(v)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_k]))} \quad (2.2)$$

donde $\mathbf{a}$ es un vector de parámetros de atención y $\parallel$ denota concatenación. Este diseño es especialmente apropiado para grafos de clúster *Kubernetes*, donde una arista de privilegio (*privilege_reach*) debe pesar más que una arista de proximidad de directorio

(*dir-proximity*). El modelo *How Powerful are GNNs?* de Xu, Hu, Leskovec, y Jegelka 2019 establece que la capacidad discriminativa de las GNNs está acotada por el test de isomorfismo de Weisfeiler-Lehman, y que la suma de vecinos maximiza el poder expresivo. Esta intuición motiva el uso de *pooling* combinado de media y máximo en la Capa 2.

2.5. GNNs aplicadas a ciberseguridad

La aplicación de GNNs a la detección de amenazas es un campo emergente con rápido crecimiento. Bilot, Madhoun, Agha, y Zouaoui 2023 proporcionan la primera revisión exhaustiva de los enfoques basados en GNN para la detección de intrusiones en red (IEEE Access, vol. 11, pp. 49114–49139), identificando tres paradigmas: detección de anomalías en grafos de flujo, clasificación de nodos en grafos de comunicación, y clasificación de grafos de comportamiento por muestra. Una segunda revisión publicada en *Computers and Security* en 2024 (Zhong, Lin, Zhang, y Xu, 2024) amplía esta taxonomía e identifica que las publicaciones sobre detección de anomalías con GNNs crecieron de 11 en 2022 a 39 en 2024, evidenciando el interés creciente del área.

E-GraphSAGE (Lo, Layeghy, Sarhan, Gallagher, y Portmann, 2022) propone representar flujos de red como grafos de transacción para clasificación de intrusiones en IoT, alcanzando tasas de detección superiores a los clasificadores tabulares clásicos en el dataset UNSW-NB15. La analogía con el presente trabajo es directa: los manifiestos *Kubernetes* son los nodos y las relaciones de privilegio y movimiento lateral son las aristas del grafo de amenaza. Una diferencia clave es que *E-GraphSAGE* modela flujos de tráfico en tiempo real, mientras que este trabajo modela configuraciones declarativas estáticas, lo que permite operar antes del despliegue.

Li, Zeng, Chen, y Liang 2022 construyen grafos de conocimiento de técnicas de ataque a partir de informes de inteligencia de amenazas (CTI), demostrando que el razonamiento estructural sobre relaciones entre técnicas mejora la predicción de campañas de ataque. Este enfoque comparte con el presente trabajo la intuición central de que el grafo de relaciones entre recursos es más informativo que la suma de sus nodos individuales. Aunque opera sobre conocimiento de ataques pasados (retrospectivo) y no sobre configuraciones declarativas (preventivo), la arquitectura basada en grafos de conocimiento inspiró el diseño de los tipos de arista semánticamente diferenciados del presente sistema.

Una limitación reconocida en la literatura es la escasez de conjuntos de datos de grafos

etiquetados de alta calidad para ciberseguridad. Tanto Bilot y cols. 2023 como Zhong y cols. 2024 identifican la construcción de benchmarks estandarizados como el principal obstáculo para el avance del área, lo que refuerza la contribución del presente trabajo al generar un conjunto de datos original de 96 grafos de clúster con etiquetas de cadena de ataque.

2.6. Detección de cadenas de ataque y grafos de ataque en Kubernetes

La noción de cadena de ataque (*kill chain*) fue formalizada originalmente por Hutchins y cols. 2011 para describir las fases secuenciales de una intrusión avanzada: reconocimiento, weaponización, entrega, explotación, instalación, comando y control, y acción. En el contexto de *Kubernetes*, esta cadena se concreta de forma más compacta: requiere como mínimo (1) un pod con capacidad de escape al anfitrión (*TRUE_HOST_PID*, *privileged: true*, *hostPath* montado, etc.) y (2) un mecanismo de movimiento lateral hacia otros recursos del clúster (token de cuenta de servicio con permisos excesivos, montaje de secretos en el manifiesto, etc.).

En los últimos años han surgido dos herramientas de código abierto que abordan la detección de rutas de ataque en *Kubernetes* mediante grafos: *KubeHound* (Datadog Security Labs, 2023) e *IceKube* (WithSecure Labs, 2023).

KubeHound (Datadog, 2023) construye un grafo de ataque del clúster a partir de la consulta directa al servidor Application Programming Interface (API) de *Kubernetes*, almacena el resultado en JanusGraph y permite explorar las rutas de ataque más cortas mediante el lenguaje de consulta Gremlin. El proyecto fue motivado por la misma intuición que subyace al presente trabajo, formulada por John Lambert: «*los defensores piensan en listas, los atacantes piensan en grafos*» (Datadog Security Labs, 2023). *KubeHound* cubre un amplio conjunto de ataques y ha sido adoptado por equipos de seguridad en producción.

IceKube (WithSecure/ReverseSecLabs, 2023) sigue un enfoque análogo, basado en Neo4j, y permite definir rutas de ataque mediante consultas Cypher. Implementa actualmente 25 técnicas de ataque como relaciones en el grafo y puede encontrar rutas desde cualquier identidad de bajo privilegio hasta *cluster-admin*.

La diferencia fundamental entre estas herramientas y *kubescan* es de paradigma: *KubeHound* e *IceKube* operan sobre un **clúster activo**, requiriendo acceso con privilegios

elevados a `kubect1`. *kubescan*, en cambio, opera sobre **manifiestos YAML estáticos** antes del despliegue, integrándose en el pipeline CI/CD como un control de seguridad *shift-left*. Además, ambas herramientas utilizan búsqueda de rutas mediante consultas ad-hoc en bases de datos de grafos, sin componente de aprendizaje automático supervisado; *kubescan* es el primer sistema, según la revisión bibliográfica realizada en este trabajo (Sección 2.7), que combina análisis estático de manifiestos con GNNs para la predicción de cadenas de ataque como tarea de clasificación supervisada.

2.7. Síntesis y posicionamiento

El análisis de la literatura permite identificar tres carencias fundamentales que motivan el presente trabajo. En primer lugar, las herramientas de análisis estático existentes (Checkov, kube-linter, Kubescape) operan a nivel de manifiesto individual y no modelan las relaciones estructurales entre recursos del clúster que habilitan las cadenas de ataque. En segundo lugar, los conjuntos de datos de referencia para seguridad de *Kubernetes* son escasos, de pequeño tamaño y desequilibrados hacia las clases de riesgo bajo, lo que dificulta el entrenamiento de clasificadores de clúster: el propio análisis del dataset de Rahman et al. confirma que `TRUE_HOST_PID` —la flag de escape más peligrosa— aparece en sólo el 0,3 % de los manifiestos reales. En tercer lugar, las herramientas de grafo de ataque más avanzadas (KubeHound, IceKube) requieren un clúster activo, lo que impide su uso como control preventivo en el pipeline CI/CD.

La Tabla 2.2 resume el posicionamiento de *kubescan* frente a las principales herramientas del estado del arte.

Tabla 2.2: Comparativa de herramientas de seguridad para Kubernetes				
Herramienta	Análisis estático	Requiere clúster	Grafos	ML supervisado
Checkov (Bridgecrew, 2021)	✓	No	No	No
kube-linter (StackRox, 2021)	✓	No	No	No
Kubescape (ARMO Ltd., 2021)	✓	Opcional	No	No
KubeHound (Datadog Security Labs, 2023)	No	Sí	✓	No
IceKube (WithSecure Labs, 2023)	No	Sí	✓	No
kubescan	✓	No	✓	✓

Fuente: elaboración propia.

El sistema *kubescan* presentado en este trabajo responde a las tres carencias identificadas: adopta una arquitectura de grafo heterogéneo con tipos de arista semánticamente significativos para el dominio *Kubernetes*, combina el análisis estático (Capa 1) con el razonamiento estructural (Capa 2) en un *ensemble* optimizado (Capa 3), y contribuye un conjunto de datos de 96 grafos de clúster con etiquetas de cadena de ataque derivadas de reglas de seguridad explícitas y auditables. Al operar sobre manifiestos YAML sin requerir acceso al clúster, *kubescan* se posiciona como un control preventivo integrable en pipelines CI/CD, complementario a —y no sustituto de— las herramientas de análisis de clúster activo como KubeHound.

3. Objetivos y Metodología

Este capítulo define los objetivos del trabajo y describe la metodología seguida para alcanzarlos. En primer lugar se formula el objetivo general y los objetivos específicos, todos ellos con criterios SMART cuantificables. A continuación se describen las cinco fases del proceso de desarrollo, la estrategia de partición y validación de datos, y la justificación de las métricas de evaluación adoptadas.

3.1. Objetivo general

Diseñar, implementar y evaluar un sistema automático de predicción de cadenas de ataque en infraestructuras *Kubernetes* que complemente los enfoques de análisis estático por manifiesto individual con el contexto estructural del clúster, mediante la combinación de un clasificador de manifiestos individuales (*Random Forest*) con una red neuronal de grafos (*Graph Attention Network*) y optimización de ensemble mediante Algoritmo Genético. El sistema opera principalmente sobre manifiestos YAML estáticos, sin requerir acceso a un clúster activo, si bien incorpora opcionalmente una vía de consulta directa al clúster (OE6).

3.2. Objetivos específicos

- OE1.** Construir un conjunto de datos de manifiestos *Kubernetes* etiquetados que combine múltiples fuentes públicas, contenga al menos 2.000 manifiestos, y alcance una cobertura mínima del 2 % en la flag de escape `TRUE_HOST_PID`, superando el 0,3 % de los repositorios reales no aumentados.
- OE2.** Desarrollar una Capa 1 de clasificación de manifiestos individuales basada en *Random Forest* con F1 macro superior a 0,85, que produzca *risk scores* calibrados para usar como características de nodo en la Capa 2.
- OE3.** Diseñar un esquema de grafo de clúster con tipos de arista semánticamente significativos para el dominio *Kubernetes*: proximidad de directorio, alcance de privilegio, movimiento lateral mediante cuenta de servicio, co-namespace semántico y escalada Role-Based Access Control (Control de Acceso Basado en Roles) (RBAC).

- OE4.** Desarrollar una Capa 2 de clasificación de clústeres basada en GAT que alcance Precisión@5 superior a 0,70 (media de validación cruzada de 5 pliegues) en la detección de cadenas de ataque.
- OE5.** Implementar una Capa 3 que optimice mediante Algoritmo Genético una función objetivo combinada de Precisión@5 y falsos positivos de clústeres limpios, garantizando False Positive Rate (Tasa de Falsos Positivos) (FPR)_{clean} = 0 sobre el conjunto de validación y manteniendo una Precisión@5 competitiva con la del mejor componente individual.
- OE6.** Empaquetar el sistema completo como una herramienta de línea de comandos (*kubescan*) con soporte para análisis de directorios locales y, opcionalmente, consulta directa al API de *Kubernetes* mediante *kubectrl*; instalable en entornos Python 3.10+ en menos de 60 segundos (`pip install`, caché de paquetes vacía, conexión de banda ancha).

3.3. Metodología

El trabajo sigue un proceso iterativo de cinco fases que combina el paradigma de desarrollo de *software* (requisitos, diseño, implementación, prueba) con el proceso de ciencia de datos (adquisición, preparación, modelado, evaluación):

Fase 1 — Adquisición de datos. Descarga y normalización de manifiestos *Kubernetes* de cuatro fuentes públicas complementarias: el conjunto de datos de Rahman et al. (Rahman y cols., 2023) (repositorios reales etiquetados), BishopFox BadPods (Art, 2021) (pods intencionadamente vulnerables), *Kubernetes Goat* (Madhuakula, 2020) (escenarios de vulnerabilidad) y repositorios de seguridad adicionales (CTFs, laboratorios y *fixtures*). El criterio de inclusión de repositorios adicionales fue la presencia de al menos una flag de escape activa (`HOSTPATH_MOUNT`, `TRUE_HOST_PID`, etc.) en al menos un manifiesto del directorio.

Fase 2 — Extracción de características y construcción del grafo. Extracción de 28 características binarias por manifiesto mediante análisis estático multi-herramienta (Sección 4.9) y construcción del grafo de clúster con cinco tipos de arista (Tabla 4.2). En esta fase se realizó también la validación de cobertura de la flag `HOSTPATH_MOUNT`, documentada en la Sección 5.4.

Fase 3 — Entrenamiento de la Capa 1. Entrenamiento de un clasificador *Random Forest* mediante validación cruzada de 5 pliegues para predecir la presencia de misconfiguraciones individuales y generar *risk scores* calibrados, con los hiperparámetros fijos detallados en la Sección 4.5.

Fase 4 — Entrenamiento de la Capa 2. Entrenamiento de una GAT de 3 capas con agrupamiento de grafos (*graph pooling*) y validación cruzada estratificada de 5 pliegues, con aumentación de datos sobre los grafos de entrenamiento para mitigar la escasez de ejemplos de cadena de ataque.

Fase 5 — Optimización de ensemble y evaluación. Optimización de los pesos del ensemble de tres componentes (RF, GNN, señal de escape) mediante Algoritmo Genético sobre el conjunto de validación, y evaluación final sobre el conjunto de test reservado desde el inicio y no utilizado en ninguna fase de entrenamiento ni selección de hiperparámetros.

3.4. Estrategia de partición y validación de datos

La estrategia de partición de datos difiere entre la Capa 1 (datos tabulares) y la Capa 2 (datos de grafos), dado que sus tamaños y distribuciones son distintos. El tamaño final de cada conjunto depende del proceso de adquisición y construcción descrito en las Secciones 4.3 y 4.4 del Capítulo 4.

Para la Capa 1, el conjunto de manifiestos se divide en un conjunto de entrenamiento (80 %) y un conjunto de test (20 %), con estratificación por la variable objetivo para mantener la proporción de clases en ambos subconjuntos. La evaluación del modelo se realiza mediante validación cruzada de 5 pliegues sobre el conjunto de entrenamiento para la selección de hiperparámetros, y el conjunto de test se reserva exclusivamente para la evaluación final.

Para la Capa 2, los grafos de clúster se evalúan mediante validación cruzada estratificada de 5 pliegues, con estratificación por etiqueta de clúster (*clean, isolated, attack_chain*). En cada pliegue, la aumentación de datos se aplica **exclusivamente** sobre los grafos de entrenamiento del pliegue, nunca sobre los de validación, evitando así la fuga de datos. La estrategia de validación cruzada es la estándar para conjuntos de grafos pequeños (Hamilton, 2020), donde no hay suficientes muestras para una partición train/val/test clásica sin comprometer la representatividad de cada subconjunto.

3.5. Métricas de evaluación

La selección de métricas de evaluación responde a los requisitos operacionales del sistema y a las características del dataset.

F1 macro para la Capa 1 y la clasificación por clases de la Capa 2: la F1 macro calcula la media no ponderada de la F1 de cada clase, otorgando igual importancia a las clases minoritarias (misconfiguraciones críticas) que a las mayoritarias (manifiestos seguros). Esta propiedad es fundamental en datasets de seguridad desequilibrados, donde la exactitud (*accuracy*) sería una métrica engañosa —un clasificador que predice siempre «seguro» obtendría una exactitud del 64,2 % pero F1 macro del 39,1 %.

AUC-ROC para el modelo binario: mide la capacidad del clasificador para separar las dos clases independientemente del umbral de decisión, lo que permite al operador ajustar la sensibilidad según las necesidades del entorno.

Precisión@k (P@k) para la Capa 2: en lugar de evaluar la clasificación por umbral, P@k mide la fracción de clústeres de cadena de ataque en las primeras k posiciones del ranking por puntuación de riesgo. Esta métrica está directamente alineada con el caso de uso operacional: un equipo de seguridad que audita los k clústeres de mayor riesgo en su infraestructura quiere que todos sean cadenas de ataque reales, no falsas alarmas. La elección de $k = 5$ refleja la capacidad de revisión manual típica de un equipo de seguridad reducido en una jornada laboral.

FPR_clean (tasa de falsos positivos sobre clústeres limpios): mide la fracción de clústeres clasificados como de alto riesgo que en realidad son limpios. Un $\text{FPR_clean} = 0$ significa que ningún clúster limpio aparece entre los de mayor puntuación, eliminando las investigaciones innecesarias.

4. Diseño e Implementación

Este capítulo describe el diseño completo del sistema *kubescan* y su implementación como herramienta de software distribuible. Se presentan, en primer lugar, los requisitos funcionales y no funcionales que guiaron las decisiones de diseño; a continuación se detalla la arquitectura de tres capas del sistema, los procesos de construcción del conjunto de datos y del grafo de clúster, y los algoritmos de entrenamiento de cada capa; finalmente se describe la interfaz de línea de comandos resultante. La implementación completa se encuentra disponible en el repositorio indicado en el Apéndice B.

4.1. Identificación de requisitos

El diseño de *kubescan* parte de un análisis de requisitos derivado tanto de las limitaciones identificadas en el estado del arte (Sección 2.7) como de los vectores de ataque documentados por la NSA y la Cybersecurity and Infrastructure Security Agency (CISA) (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022).

4.1.1. Requisitos funcionales

RF-1 — Detección de misconfiguraciones individuales. El sistema debe clasificar cada manifiesto YAML como seguro o *misconfigured*, produciendo además un índice de severidad en tres niveles (limpio, bajo–medio, alto–crítico).

RF-2 — Detección de cadenas de ataque. El sistema debe clasificar un conjunto de manifiestos que describe un clúster en una de tres categorías: *clean*, *isolated* (misconfiguraciones sin cadena viable) o *attack_chain* (ruta de escalada de privilegios hasta movimiento lateral).

RF-3 — Interfaz de línea de comandos. El sistema debe exponer un comando `kubescan scan <dir>` para análisis de directorios locales y un comando `kubescan live` para análisis de clústeres activos mediante *kubectrl*.

RF-4 — Formatos de salida. El sistema debe producir informes en formato texto legible y en formato JSON estructurado para integración con herramientas externas.

RF-5 — Priorización de riesgos. El sistema debe producir un ranking de clústeres

por puntuación de riesgo que maximice la Precisión@5 — colocando las cadenas de ataque reales en las primeras posiciones.

4.1.2. Requisitos no funcionales

RNF-1 — Trazabilidad. Todas las reglas de detección de nodos de escape y movimiento lateral deben ser explícitas, auditables y derivadas de fuentes normativas documentadas (Art, 2021; National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022).

RNF-2 — Reproducibilidad. Los modelos entrenados deben almacenarse como *checkpoints* e incluirse en el paquete, de forma que el análisis sea idempotente dado el mismo directorio de entrada.

RNF-3 — Instalabilidad. El sistema debe ser instalable mediante `pip install -e kubescan/` sin dependencias de compiladores externos.

RNF-4 — Latencia. El análisis de un clúster de hasta 500 manifiestos debe completarse en menos de 60 segundos en hardware de gama media (CPU).

4.2. Arquitectura general del sistema

El sistema *kubescan* implementa una arquitectura de tres capas que transforma un directorio de manifiestos YAML en una puntuación de riesgo de cadena de ataque (Figura 4.1):

1. **Capa 1 (Random Forest):** extrae 25 características binarias por manifiesto y produce un *risk_score* $\in [0, 1]$.
2. **Capa 2 (GAT):** modela el clúster como un grafo donde los nodos son manifiestos y las aristas representan relaciones de seguridad; produce una probabilidad de cadena de ataque $\in [0, 1]$.
3. **Capa 3 (Ensemble GA):** combina las señales de las dos capas anteriores con una señal binaria de escape según la ecuación:

$$\text{score}(C) = w_{\text{rf}} \cdot \overline{\text{risk}}(C) + w_{\text{gnn}} \cdot p_{\text{chain}}(C) + w_{\text{esc}} \cdot \mathbf{1}[\exists \text{ nodo de escape en } C] \quad (4.1)$$

donde los pesos w_{rf} , w_{gnn} , w_{esc} son optimizados mediante Algoritmo Genético para maximizar la Precisión@5 en el conjunto de validación.

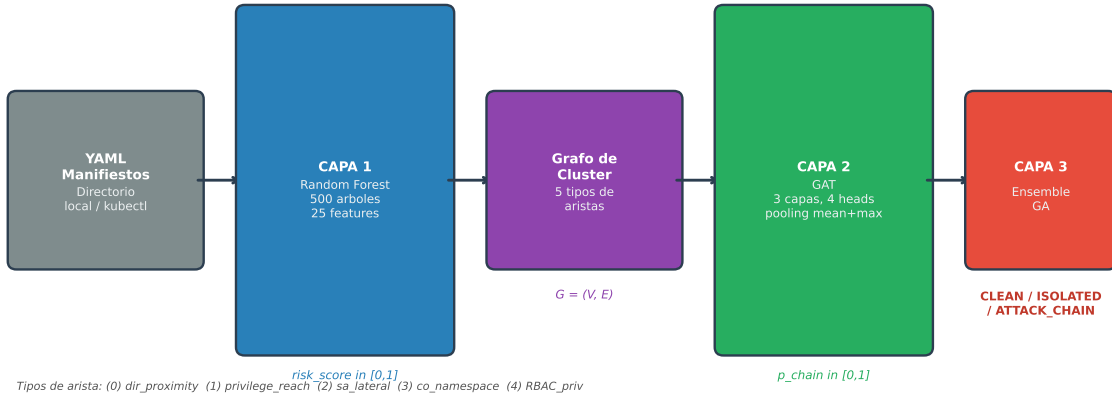


Figura 4.1: Arquitectura del sistema *kubescan*: pipeline de tres capas desde manifiestos YAML de entrada hasta la clasificación de riesgo del clúster. Los *risk_scores* producidos por la Capa 1 se incorporan como atributo nodal 26 del grafo de entrada de la Capa 2.

Fuente: elaboración propia.

4.3. Construcción del dataset

4.3.1. Fuentes de datos

El conjunto de datos tabular final contiene 2.479 filas y 58 columnas, combinando cinco fuentes (agrupadas en cuatro categorías, dado que Rahman et al. aporta dos plataformas distintas con características de distribución diferentes; ver Tabla 4.1):

La incorporación de BadPods fue esencial para mitigar la escasez de ejemplos de misconfiguraciones críticas en los repositorios reales: antes de añadir esta fuente, `TRUE_HOST_PID` aparecía en sólo el 0,3 % de las filas; tras la incorporación, la cobertura ascendió al 2,8 %. En cuanto a los conjuntos de datos de referencia existentes para intrusión en red, como UNSW-NB15 (Moustafa y Slay, 2015), se estudió su metodología de construcción como guía para el diseño del proceso de etiquetado y la estrategia de balanceo de clases, dado que presentan un desequilibrio de clases del mismo orden de magnitud que el de este trabajo (64,2 % seguros vs. 35,8 % misconfigured). El conjunto

Tabla 4.1: Composición del conjunto de datos de manifiestos Kubernetes

Fuente	Filas	Rol
Rahman et al. (Rahman y cols., 2023) — GitHub	1.510	Manifiestos reales, etiquetados
Rahman et al. (Rahman y cols., 2023) — GitLab	396	Ídem, plataforma distinta
BishopFox BadPods (Art, 2021)	128	Pods vulnerables intencionales
Kubernetes Goat (Madhuakula, 2020)	14	Escenarios de vulnerabilidad
Repositorios de seguridad adicionales	431	CTFs, <i>fixtures</i> de herramientas, labs
Total	2.479	

Fuente: elaboración propia.

GenKubeSec (Malul y cols., 2024), publicado en 2024, no estaba disponible durante el desarrollo de este trabajo; en su lugar se empleó *Checkov* (Bridgecrew, 2021) directamente sobre los ficheros YAML descargados como fuente de validación multi-herramienta equivalente.

4.3.2. Características extraídas

El extractor de características aplica 28 reglas de análisis estático a cada manifiesto YAML, organizadas en tres grupos:

- **18 flags Rahman** (subconjunto del trabajo original): `TRUE_HOST_PID`, `CAP_SYS_ADMIN`, `SEC_CONT_OVER_PRIVIL`, `INSECURE_HTTP`, etc.
- **3 features derivadas**: `cap_misuse` (OR de las dos flags de capacidades de sistema), `all_secrets` (OR de los indicadores de exposición de secretos), `total_misconfigs` (suma de todas las flags activas).
- **7 features extendidas**: `NO_RUN_AS_NON_ROOT`, `IMAGE_USES_LATEST`, `SA_AUTOMOUNT_TOKEN`, `HOSTPATH_MOUNT`, etc.; imputadas mediante *Checkov* para el 47,4 % de filas con YAML descargado, y mediante la mediana de columna para el resto.

La Capa 2 (GNN) consume únicamente las 18 flags Rahman y las 7 features extendidas (25 en total, `FEATURE_COLS` en `yaml_parser.py`) como vector de nodo, excluyendo

las 3 features derivadas —agregados calculados a partir de las otras 25 que no aportan información estructural adicional al grafo—; ver Sección 4.9.

Tres features presentan varianza casi nula pero se mantienen en el entrenamiento (con importancia residual o nula, Sección 5.1.2): `SECCOMP_UNCONFINED` (15/2.479 activa), `VALID_TAINT_SECRET` (0/2.479 activa) y `NO_NETWORK_POLICY` (2.478/2.479 activa, sin poder discriminativo).

4.4. Construcción del grafo de clúster

Cada directorio de manifiestos se representa como un grafo dirigido $G = (V, E)$ donde $|V|$ es el número de manifiestos del clúster y E contiene aristas de cinco tipos semánticamente distintos (Tabla 4.2). La Figura 4.2 ilustra un ejemplo con cuatro nodos que incluye los tres tipos de nodo posibles: escape, movimiento lateral y limpio.

Tabla 4.2: Tipos de aristas en el grafo de clúster

Tipo	Código	Dirección	Criterio
Proximidad de directorio	0	Bidireccional	Mismo subdirectorio YAML
Alcance de privilegio	1	Dirigida (escape→todos)	Flag de escape activa
Movimiento lateral SA	2	Dirigida (lateral→todos)	Flag de movimiento lateral
Co-espacio de nombres (<code>SEMANTIC_NS</code>)	3	Bidireccional	Mismo <i>namespace</i> K8s
Escalada RBAC	4	Dirigida (elevado→todos)	Service Account (Cuenta de Servicio de Kubernetes) (SA) vinculado a rol privilegiado

Fuente: elaboración propia.

Cada nodo v_i lleva un vector de características $\mathbf{x}_i \in \mathbb{R}^{26}$: índices 0–24 corresponden a las 25 features binarias de la Capa 1 y el índice 25 contiene el *risk_score* producido por el modelo *Random Forest*.

Como se observa en la Figura 4.2, el nodo rojo (pod-A) tiene flags de escape activas y emite aristas de tipo 1 (*privilege_reach*) hacia todos los demás nodos del clúster. El nodo

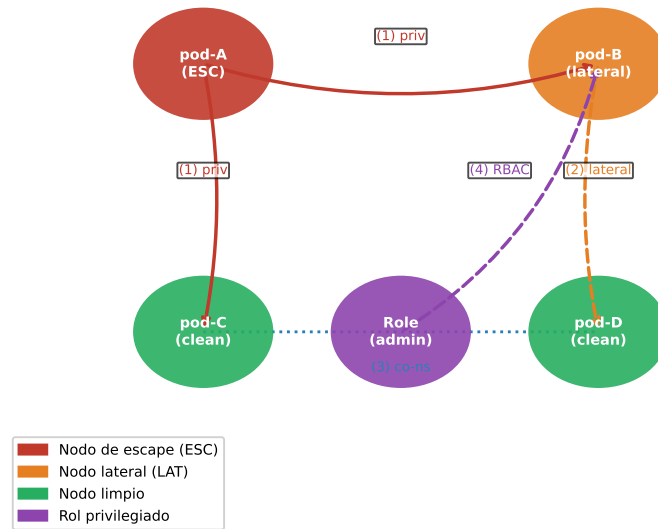


Figura 4.2: Ejemplo de grafo de clúster Kubernetes con cuatro nodos, mostrando los tres tipos de nodo (escape, movimiento lateral y limpio) y las aristas dirigidas que cada uno origina.

Fuente: elaboración propia.

aranja (pod-B) tiene flags de movimiento lateral y emite aristas de tipo 2 (*sa_lateral*). La arista de tipo 4 (RBAC) representa la vinculación de un *ServiceAccount* a un rol privilegiado.

El dataset de grafos final comprende 96 grafos originales (25 clean, 46 isolated, 25 attack_chain) y 471 grafos totales tras la aumentación de datos aplicada exclusivamente en el conjunto de entrenamiento. Los 14 nuevos clústeres de cadena de ataque provienen de 24 repositorios de seguridad adicionales (CTFs, laboratorios y *fixtures* de herramientas de análisis estático) incorporados tras la fase inicial de adquisición de datos.

4.4.1. Detección de escape y movimiento lateral

La clasificación de nodos utiliza dos conjuntos de flags definidos en el código como constantes inmutables (**frozenset**). Un nodo se clasifica como de **escape** si tiene activa al menos una de las siguientes 8 flags: `TRUE_HOST_PID`, `TRUE_HOST_IPC`, `TRUE_HOST_NET`, `DOCKERSOCK_PATH`, `CAP_SYS_ADMIN`, `CAP_SYS_MODULE`, `SEC_CONT_OVER_PRIVIL` o `HOSTPATH_MOUNT`. Este último vector fue incorporado durante el desarrollo al detectar que *badpods_hostpath* —que monta el sistema de ficheros del anfitrión— era

incorrectamente etiquetado como *isolated* al no tener ninguna otra flag de escape activa. Con la corrección, el número de clústeres de cadena de ataque ascendió de 13 a 15.

Un nodo se clasifica como de **movimiento lateral** si tiene activa al menos una de las siguientes 4 flags: `SA_AUTOMOUNT_TOKEN` (token de cuenta de servicio auto-montado), `USES_DEFAULT_SA` (usa la cuenta de servicio por defecto del *namespace*), `WITHIN_MANIFEST_SECRET` (secreto referenciado en el manifiesto) o `ALLOW_PRIVI` (contenedor con capacidad de escalada). Las aristas de movimiento lateral (tipo 2) se construyen de forma análoga a las de escape: se emite una arista dirigida desde cada nodo lateral hacia todos los demás, pero únicamente si no existe ya una arista de mayor prioridad (escape o RBAC).

4.4.2. Detección de privilegio RBAC

La construcción de aristas RBAC (tipo 4) requiere dos pasadas sobre el conjunto de manifiestos. La primera pasada identifica los roles privilegiados: aquellos cuyo nombre está en el conjunto `{cluster-admin, admin, edit}` o cuyas reglas contienen verbos de escalada `{*, escalate, bind, impersonate}`. La segunda pasada recorre los objetos *RoleBinding* y *ClusterRoleBinding*: sólo cuando el campo *roleRef* referencia un rol privilegiado identificado en la primera pasada se emite una arista RBAC desde el pod que usa esa cuenta de servicio hacia todos los demás nodos del clúster.

4.5. Capa 1: Random Forest

4.5.1. Modelo binario

Se entrena un *RandomForestClassifier* de *scikit-learn* (Pedregosa y cols., 2011) con los hiperparámetros siguientes:

- `n_estimators = 500`
- `max_features = sqrt` (clasificación estándar)
- `min_samples_leaf = 2` (evita hojas de muestra única)
- `class_weight = balanced` (compensa ratio 1,79:1)
- `random_state = 42`

El modelo se entrena sobre 1.983 muestras (80 % del total) y se evalúa en 496 muestras de test no vistas durante el entrenamiento. La probabilidad predicha para la clase *misconfigured* (clase 1) se usa como *risk_score* del nodo en el grafo.

4.5.2. Modelo de severidad

Además del modelo binario, se entrena un clasificador de severidad de tres clases (*clean*, *low-medium*, *high-critical*) para ofrecer información más granular al operador de seguridad. Este modelo no interviene en el cálculo del grafo; se incluye como salida adicional de la Capa 1.

4.6. Capa 2: Graph Attention Network

4.6.1. Arquitectura

El modelo GAT (Veličković y cols., 2018) implementado tiene la siguiente estructura, con las dimensiones exactas verificadas en `gat_encoder.py`:

1. **Embedding de tipo de arista:** `Embedding(5, 8)`. Los cinco tipos de arista (proximidad de directorio, alcance de privilegio, lateral por *ServiceAccount*, espacio de nombres y RBAC) son categóricos: codificarlos como un escalar continuo implicaría un orden espurio (el tipo 4 «valdría» cuatro veces el tipo 1). Cada tipo se proyecta a un vector aprendido de 8 dims que `GATConv` consume como atributo de arista (*edge_dim=8*).
2. **Proyección inicial:** `Linear(26, 256) + LayerNorm(256) + ReLU + Dropout(0.3)`. Proyecta el vector de nodo de 26 dims a 256 dims (64×4 cabezas).
3. **Capas GAT 0 y 1** (*concat=True*): `GATConv(256, 64, heads=4)` con *edge_dim=8*; salida 256 dims (64×4); seguidas de `LayerNorm`, ELU, conexión residual y `Dropout(0.3)`.
4. **Capa GAT 2** (*concat=False, heads=1*): `GATConv(256, 64, heads=1)`; salida 64 dims (una sola cabeza de atención que produce directamente el embedding final del nodo, sin concatenación); seguida de `LayerNorm`, Exponential Linear Unit (ELU), conexión residual y `Dropout(0.3)`.

5. **Global pooling:** `global_mean_pool` y `global_max_pool` sobre los 64-dim embeddings de todos los nodos; concatenación $\rightarrow$ 128 dims (64 + 64).
6. **Clasificador Multi-Layer Perceptron (Perceptrón Multicapa) (MLP):**
`Linear(128, 64) + Rectified Linear Unit (ReLU) + Dropout(0.3) + Linear(64, 3)`.

La elección de GAT sobre GCN responde a que el mecanismo de atención permite al modelo aprender qué aristas son más informativas para la señal de ataque: una arista de *privilege_reach* debe recibir mayor peso que una arista de proximidad de directorio. El *pooling* combinado de media y máximo captura dos señales complementarias: la media refleja la postura de seguridad promedio del clúster, mientras que el máximo identifica el nodo más peligroso, ambas necesarias para detectar cadenas de ataque que pueden estar concentradas en un único nodo crítico.

4.6.2. Entrenamiento

El modelo se entrena con *PyTorch Geometric* (Fey y Lenssen, 2019) bajo los siguientes hiperparámetros: optimizador AdamW con tasa de aprendizaje 5×10^{-4} y *weight_decay* = 10^{-4} ; *scheduler* CosineAnnealingLR con `T_max` = 300 y $\eta_{\min} = 10^{-5}$; función de pérdida CrossEntropyLoss con pesos de clase calculados por frecuencia inversa; *gradient clipping* con norma máxima 2,0; *early stopping* con paciencia de 50 épocas; y validación cruzada estratificada de 5 pliegues.

Los pesos de clase se calculan en cada pliegue de entrenamiento mediante la siguiente fórmula implementada en `train_gnn.py`:

$$w_i = \frac{1/n_i}{\sum_{j=0}^{K-1} 1/n_j} \cdot K \quad (4.2)$$

donde n_i es el número de grafos de entrenamiento de la clase i y $K = 3$ es el número de clases. Esta fórmula normaliza los pesos para que su media sea 1,0, equivalente al comportamiento de `class_weight=balanced` de *scikit-learn*. Un detalle importante es que los pesos se calculan **después** de la aumentación de datos: como la aumentación genera 15 variantes por grafo de cadena de ataque, la clase *attack_chain* pasa de ser la minoría a ser la mayoría en el conjunto de entrenamiento (aproximadamente 250 grafos aumentados

vs. 20 *clean* y 37 *isolated* en el conjunto de entrenamiento de cada pliegue, tras excluir las variantes de los clústeres de validación y test). La Ecuación 4.2 se auto-adapta a esta nueva distribución asignando mayor peso a *clean* e *isolated*, y menor peso a *attack_chain*, compensando el efecto de la sobre-representación creada por la aumentación.

4.6.3. Aumentación de datos

Para mitigar la escasez de ejemplos de cadena de ataque (15 de 82 grafos en la primera fase), se aplican tres estrategias de aumentación exclusivamente sobre los grafos de entrenamiento: *edge dropout* (eliminación aleatoria del 20 % de aristas de los tipos 0 y 3, los menos críticos para la señal de ataque); *feature masking* (puesta a cero aleatoria del 15 % de valores en el vector de nodo); y *subgraph sampling* (extracción de un subgrafo inducido por el 70 % de nodos del grafo original, manteniendo los nodos de escape; cada subgrafo se valida contra la regla de etiquetado de cadena y, si deja de cumplirla, se descarta en favor de una variante sobre el grafo completo, evitando introducir ruido de etiqueta). Cada grafo de cadena de ataque origina 15 variantes aumentadas, resultando en 375 grafos adicionales (25 originales $\times$ 15 variantes), para un total de 471 grafos con aumentación.

La asignación de variantes a particiones es **consciente de grupos** (*group-aware*): una variante aumentada solo puede entrar en una partición de entrenamiento si su grafo base pertenece a esa misma partición. Las variantes derivadas de clústeres de validación o de test se excluyen del entrenamiento por completo, ya que entrenar con casi-duplicados de un grafo de evaluación constituiría fuga de datos. Adicionalmente, los 15 clústeres de test se mantienen fuera de los pliegues de validación cruzada: los pliegues particionan únicamente los 81 grafos originales restantes, de modo que ni los modelos de pliegue ni las predicciones *out-of-fold* empleadas para ajustar los pesos del *ensemble* (Capa 3) observan ningún clúster de test, directa ni indirectamente.

4.7. Capa 3: Ensemble con Algoritmo Genético

Los pesos ($w_{\text{rf}}, w_{\text{gnn}}, w_{\text{esc}}$) de la Ecuación 4.1 son optimizados mediante un Algoritmo Genético (Holland, 1975) con 60 individuos codificados como pares $(w_{\text{gnn}}, w_{\text{esc}}) \in [0, 1]^2$, con w_{rf} derivado como $1 - w_{\text{gnn}} - w_{\text{esc}}$ (la representación 2D garantiza que la suma de los tres pesos sea siempre 1). La función objetivo es:

$$F = 0,7 \cdot P@5 + 0,3 \cdot (1 - \text{FPR}_{\text{clean}}) \quad (4.3)$$

La selección se realiza mediante torneo de tamaño 3; el cruce es aritmético (media de los dos padres); la mutación aplica perturbación gaussiana con $\sigma = 0,08$ y tasa de mutación del 15 %, seguida de re-proyección al dominio válido ($w_{\text{rf}} \geq 0$); y el criterio de parada es 150 generaciones. Como *baseline* determinístico, se ejecuta previamente una búsqueda en rejilla 2D con paso 1/20 (231 evaluaciones); la comparación de su óptimo con el resultado del Genetic Algorithm (Algoritmo Genético) (GA) se discute en la Sección 5.3.

La señal de escape se define como un indicador binario que vale 1,0 si al menos un nodo del clúster tiene alguna flag de escape activa, y 0,0 en caso contrario. Esta formulación evita la dilución que produciría una fracción cuando la mayoría de los nodos del clúster son limpios.

La asignación de clase final a partir de la puntuación *ensemble_score* se realiza mediante los siguientes umbrales fijos, definidos en `ga_ensemble.py`:

- *ensemble_score* $\geq 0,60 \Rightarrow \text{ATTACK_CHAIN}$
- *ensemble_score* $\geq 0,30 \Rightarrow \text{ISOLATED_MISCONFIG}$
- De lo contrario $\Rightarrow \text{CLEAN}$

Estos umbrales son fijos y no se reoptimizan durante el entrenamiento del GA, que optimiza únicamente los pesos w_i de la Ecuación 4.1. Permiten al operador interpretar directamente la puntuación: cualquier clúster con *score* $\geq 0,60$ requiere revisión inmediata.

4.8. Implementación de kubescan

El sistema se distribuye como un paquete Python instalable mediante `pip install -e kubescan/`. La estructura del paquete sigue la convención *src-layout* de *setuptools*: el código fuente reside en `kubescan/src/kubescan/` y los módulos se organizan en tres submódulos: `model/` (clasificadores RF, GAT y ensamblador GA), `utils/` (*parser* YAML y constructor de grafos) y `cli.py` (interfaz de línea de comandos basada en *Click*).

El punto de entrada principal expone dos comandos. El primero, `kubescan scan <dir>`, analiza un directorio local de manifiestos YAML y emite un informe de riesgo

en formato texto o JavaScript Object Notation (JSON). El segundo, *kubescan live*, consulta el API de *Kubernetes* mediante *kubectrl*, vuelca los recursos de *workloads* y RBAC en un directorio temporal y ejecuta el *pipeline* completo sobre ellos. La resolución de los pesos del modelo sigue la cadena: argumento `--checkpoints-dir` → variable de entorno `KUBESCAN_CHECKPOINTS` → enlace simbólico `kubescan/checkpoints/trained/` → `FileNotFoundError`.

4.9. Tabla de características del sistema

Las Tablas 4.3 y 4.4 describen las 28 características utilizadas por la Capa 1, organizadas por grupo y con indicación de si son relevantes para la detección de escape (ESC) o de movimiento lateral (LAT) en el grafo de la Capa 2. De estas 28, las 25 que no son derivadas (18 Rahman + 7 extendidas) también forman el vector de nodo de la Capa 2.

Las 8 flags de escape (ESC, `ESCAPE_FLAGS` en `graph_builder.py`) son `TRUE_HOST_PID`, `TRUE_HOST_IPC`, `TRUE_HOST_NET`, `DOCKERSOCK_PATH`, `CAP_SYS_ADMIN`, `CAP_SYS_MODULE`, `SEC_CONT_OVER_PRIVIL` y `HOSTPATH_MOUNT`. Las 4 flags de movimiento lateral (LAT, `LATERAL_FLAGS`) son `SA_AUTOMOUNT_TOKEN`, `USES_DEFAULT_SA`, `WITHIN_MANIFEST_SECRET` y `ALLOW_PRIVI`. Las 7 características extendidas están imputadas mediante *Checkov* para el 47,4 % de filas con YAML disponible y mediante la mediana de columna para el resto.

4.10. Calidad de software y proceso de validación

El desarrollo de *kubescan* sigue prácticas de ingeniería de software orientadas a la producción. La calidad del código se garantiza mediante cuatro herramientas integradas en un *pipeline* de pre-commit y CI/CD: (1) *ruff*, linter y formateador de código Python con reglas de estilo y seguridad; (2) *mypy* en modo estricto, para comprobación estática de tipos en todos los módulos del paquete; (3) *pytest*, para la suite de tests unitarios e de integración del paquete; y (4) *pre-commit hooks*, que aplican automáticamente *ruff* y *mypy* en cada commit.

La suite de tests cubre los siguientes componentes: el parser de YAML

(`utils/yaml_parser.py`), la construcción del grafo de clúster

(`utils/graph_builder.py`), el clasificador RF (`model/rf_classifier.py`) y el

ensamblador GA (`model/ga_ensemble.py`). Los tests de integración verifican el *pipeline*

completo de extremo a extremo: dado un directorio de manifiestos de prueba con flags de

Tabla 4.3: Características del clasificador RF — grupo Rahman (1/2)

Nombre	Grupo	Rol en grafo	Descripción
TRUE_HOST_PID	Rahman	ESC	Comparte PID namespace del anfitrión
TRUE_HOST_IPC	Rahman	ESC	Comparte IPC namespace del anfitrión
TRUE_HOST_NET	Rahman	ESC	Comparte red del anfitrión
DOCKERSOCK_PATH	Rahman	ESC	Monta socket de Docker
CAP_SYS_ADMIN	Rahman	ESC	Capacidad <code>SYS_ADMIN</code> activa
CAP_SYS_MODULE	Rahman	ESC	Capacidad <code>SYS_MODULE</code> activa
WITHIN_MANIFEST_SECRET	Rahman	LAT	Secreto en texto plano embebido en el manifiesto
SEC_CONT_OVER_PRIVIL	Rahman	ESC	Contenedor privilegiado (<code>privileged: true</code>)
ALLOW_PRIVI	Rahman	LAT	<code>allowPrivilegeEscalation: true</code>
INSECURE_HTTP	Rahman	—	Usa HTTP sin TLS en puerto conocido
NO_SECU_CONTEXT	Rahman	—	Ausencia de <code>securityContext</code> definido
HOST_ALIAS	Rahman	—	Usa <code>hostAliases</code> para resolución DNS manual
NO_DEFAULT_NAMESPACE	Rahman	—	Despliega en el <code>namespace default</code>
NO_RESO	Rahman	—	Sin límites/ <i>requests</i> de recursos definidos
NO_ROLLING_UPDATE	Rahman	—	Sin estrategia de actualización <i>RollingUpdate</i>
SECCOMP_UNCONFINED	Rahman	—	Perfil <i>seccomp unconfined</i> (sin filtrado de <i>syscalls</i>)
VALID_TAINT_SECRET	Rahman	—	Secreto válido referenciado en <i>taint/toleration</i>
NO_NETWORK_POLICY	Rahman	—	Sin <code>NetworkPolicy</code> definida en el <code>namespace</code>

Fuente: elaboración propia.

Tabla 4.4: Características del clasificador RF — grupos derivado y extendido, y nodo (2/2)

Nombre	Grupo	Rol en grafo	Descripción
<code>cap_misuse</code>	Derivada	—	OR lógico de <code>CAP_SYS_ADMIN</code> y <code>CAP_SYS_MODULE</code>
<code>all_secrets</code>	Derivada	—	OR de los indicadores de exposición de secretos
<code>total_misconfigs</code>	Derivada	—	Suma de todas las flags activas
<code>NO_RUN_AS_NON_ROOT</code>	Extendida	—	Sin restricción de usuario no root
<code>NO_READ_ONLY_ROOT_FS</code>	Extendida	—	Sistema de ficheros raíz con escritura
<code>IMAGE_USES_LATEST</code>	Extendida	—	Imagen con <code>tag: latest</code>
<code>SA_AUTOMOUNT_TOKEN</code>	Extendida	LAT	<i>ServiceAccount</i> con token automontado
<code>USES_DEFAULT_SA</code>	Extendida	LAT	Usa la <i>ServiceAccount default</i> del <i>namespace</i>
<code>UNTRUSTED_REGISTRY</code>	Extendida	—	Imagen proveniente de un registro no confiable
<code>HOSTPATH_MOUNT</code>	Extendida	ESC	Monta ruta del sistema de ficheros del anfitrión
<code>risk_score</code>	Nodo (Capa 2)	—	Probabilidad RF clase misconfigured $\in [0, 1]$

Fuente: elaboración propia.

escape conocidas, se comprueba que la salida del comando `kubescan scan` produce el veredicto esperado (ATTACK_CHAIN) con *ensemble_score* por encima del umbral de clasificación.

5. Resultados y Evaluación

Este capítulo presenta y analiza los resultados obtenidos en cada una de las tres capas del sistema *kubescan*, seguidos de una validación cruzada con herramientas de análisis estático externas y de una evaluación funcional de la herramienta como software distribuido. Los resultados se organizan en el mismo orden que las fases de entrenamiento descritas en el Capítulo 4. Las métricas empleadas se justifican en detalle en la Sección 3.5.

5.1. Resultados de la Capa 1: Random Forest

5.1.1. Clasificación binaria

La Tabla 5.1 resume los resultados del modelo binario mediante cuatro métricas: F1 macro (media no ponderada de F1 entre las clases seguro y misconfigured, que da igual peso a ambas independientemente de su frecuencia en el dataset), exactitud (*accuracy*, porcentaje simple de aciertos), AUC-Receiver Operating Characteristic (ROC) (capacidad del modelo para separar las dos clases a cualquier umbral de decisión) y *out-of-bag score* (estimación interna de generalización que *Random Forest* calcula sobre las muestras que cada árbol no vio durante su entrenamiento, sin necesitar un conjunto de validación aparte). El clasificador alcanza una F1 macro de 0,9935 en el conjunto de test, superando el objetivo de diseño (OE2, F1 macro $\geq 0,85$) en 14,4 puntos porcentuales.

Tabla 5.1: Resultados del Random Forest — clasificación binaria

Métrica	Test	CV (5-fold)	Objetivo
F1 macro	0,9935	$0,9908 \pm 0,0059$	$> 0,85$ ✓
Exactitud (<i>accuracy</i>)	0,9940	$0,9915 \pm 0,0055$	—
AUC-ROC	0,9980	$0,9981 \pm 0,0017$	—
<i>Out-of-bag score</i>	0,9909	—	—

Fuente: elaboración propia.

La Figura 5.1 muestra la matriz de confusión sobre el conjunto de test (496 muestras). El modelo comete sólo 3 errores totales: 3 falsos positivos (manifiestos seguros clasificados como misconfigured) y 0 falsos negativos. La ausencia de falsos negativos es la propiedad más crítica para seguridad operacional: significa que ningún manifiesto con misconfiguraciones reales escapa al filtrado de la Capa 1, independientemente de cuántas flags

estén activas o de su rareza en el dataset. Los 3 falsos positivos corresponden a manifestos seguros que acumulan varias features de riesgo de severidad media (por ejemplo, `NO_RUN_AS_NON_ROOT` y `IMAGE_USES_LATEST` simultáneamente), pero que no tienen ninguna flag de escape activa; el operador de seguridad puede descartarlos rápidamente inspeccionando el campo `flags` de la salida JSON.

El AUC-ROC de 0,9980 (prácticamente 1,00) indica que el modelo puede ordenar las muestras por probabilidad de misconfiguration de forma casi perfecta a cualquier umbral de decisión. En la práctica, este valor implica que, si se ordenan los 496 manifestos de test de mayor a menor *risk_score*, los manifestos misconfigured aparecen casi todos antes que los seguros. Esta propiedad es la que habilita el uso del *risk_score* como característica nodal cuantitativa en el grafo de la Capa 2: la señal transmitida a la GNN no es un bit binario sino una probabilidad calibrada que refleja la densidad de misconfiguraciones del manifesto.

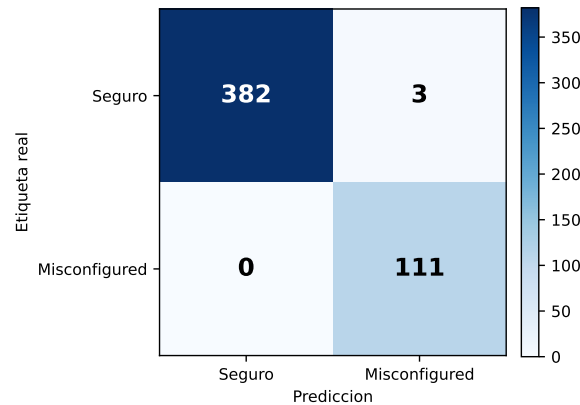


Figura 5.1: Matriz de confusión del modelo Random Forest en el conjunto de test (496 muestras). Los 3 falsos positivos son manifestos seguros con alta densidad de características de riesgo; no existe ningún falso negativo.

Fuente: elaboración propia.

5.1.2. Importancia de características

La Figura 5.2 muestra las 10 características más relevantes del modelo. La característica más discriminativa es `total_misconfigs` (32,92 % de importancia), seguida de `INSECURE_HTTP` (15,21 %) y `NO_RESO` (12,73 %). Las flags de escape críticas (`TRUE_HOST_PID`, `SEC_CONT_OVER_PRIVIL`) presentan importancias menores (<1 %) por su

rareza en el dataset, pero son identificadas correctamente cuando están presentes, como confirma la ausencia de falsos negativos en la clasificación binaria.

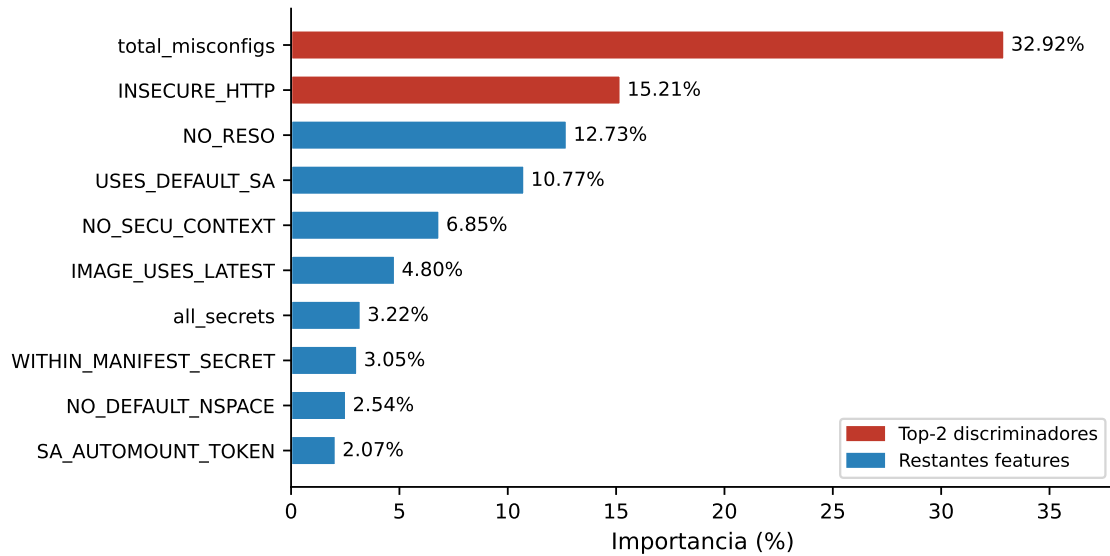


Figura 5.2: Importancia de las 10 características más relevantes del clasificador Random Forest. Las dos primeras (en rojo) concentran el 48 % de la capacidad discriminativa total del modelo.

Fuente: elaboración propia.

Un estudio de ablación eliminando `total_misconfigs` produce F1 idéntico y la misma matriz de confusión, confirmando que el modelo no depende de esta característica derivada y que no existe fuga de datos circular.

5.1.3. Modelo de severidad

El clasificador de tres clases (*clean*, *low-medium*, *high-critical*) alcanza F1 macro=0,9732 en test. La Tabla 5.2 desglosa por clase la precisión (proporción de predicciones de esa clase que son correctas), el *recall* (proporción de casos reales de esa clase que el modelo identifica), el F1 (media armónica de ambas) y el soporte (número de muestras de test en esa clase):

La clase *high-critical* (56 muestras de test, que representan manifiestos con flags de escape activas como `TRUE_HOST_PID` o `SEC_CONT_OVER_PRIVIL`) obtiene el F1 más bajo de las tres clases (0,9541), con 4 errores: 1 manifiesto crítico clasificado como *clean* y 3 clasificados como *low-medium*. El primero de estos errores es el más relevante desde el punto de vista de seguridad operacional, ya que representa un falso negativo crítico que un

Tabla 5.2: F1 por clase — modelo de severidad (3 clases, test)

Clase	Precisión	Recall	F1	Soporte
<i>clean</i>	0,9968	0,9906	0,9937	318
<i>low-medium</i>	0,9528	0,9918	0,9719	122
<i>high-critical</i>	0,9811	0,9286	0,9541	56
Macro media	0,9769	0,9703	0,9732	—

Fuente: elaboración propia.

operador podría pasar por alto; los otros errores del modelo ocurren en la frontera entre *clean* y *low-medium*, donde algunos manifiestos con pocas flags de baja severidad pueden ser sobreclasificados como *low-medium* (falsos positivos benignos desde el punto de vista del operador).

El modelo de severidad se utiliza en el informe de salida de *kubescan* para proporcionar al operador una descripción cualitativa del nivel de riesgo de cada manifiesto, complementando el *risk_score* numérico del modelo binario.

5.2. Resultados de la Capa 2: GNN

5.2.1. Evolución por fase experimental

La Tabla 5.3 muestra la evolución de los resultados de la GAT a lo largo de las cinco fases del desarrollo experimental. La Figura 5.3 ilustra visualmente la trayectoria de las cuatro primeras fases, mostrando cómo la aumentación de datos y la corrección del vector de escape *HOSTPATH_MOUNT* contribuyeron de forma independiente y complementaria. La quinta fase corrige el protocolo de particionado: las fases anteriores incluían en el entrenamiento variantes aumentadas derivadas de los clústeres de validación y test (fuga de datos por casi-duplicados), por lo que sus métricas están infladas de forma optimista. La fase final aplica el protocolo *group-aware* descrito en la Sección de diseño (las variantes acompañan a su clúster base y los clústeres de test quedan fuera de los pliegues) e incorpora el *embedding* de tipos de arista. Sus métricas son inferiores pero metodológicamente válidas, y son las que se reportan como resultado final de este trabajo.

La columna «Cobert. techo» indica en cuántos de los 5 pliegues el modelo alcanza el *techo teórico* de P@5: el valor máximo de P@5 posible dado el número de cadenas de

Tabla 5.3: Evolución de resultados de la Capa 2 (GAT, 5-fold CV). Las fases 1–4 emplean el protocolo de particionado preliminar (con fuga por variantes aumentadas) y se reportan como registro histórico; la fase 5 es el resultado final con protocolo *group-aware*.

Fase	Grafos	Cadenas	F1 macro	P@5	Cobert. techo
Línea base	82	13	$0,829 \pm 0,098$	$0,400 \pm 0,179$	2/5
Tras aumentación	277	13	$0,915 \pm 0,059$	$0,520 \pm 0,098$	5/5
Tras corrección HOST-PATH	307	15	$0,915 \pm 0,050$	$0,600 \pm 0,000$	5/5
Dataset extendido	471	25	$0,917 \pm 0,065$	$0,880 \pm 0,098$	5/5
Protocolo <i>group-aware</i> + <i>embedding</i> de aristas (final)	471	25	$0,870 \pm 0,075$	$0,720 \pm 0,098$	3/5

Fuente: elaboración propia.

ataque presentes en ese pliegue, que se alcanza cuando todas ellas quedan situadas en el top-5 del ranking.

Como se observa en la Figura 5.3, las líneas de puntos indican los objetivos de diseño (F1 $\geq 0,85$ y P@5 $\geq 0,70$) y las barras de error representan la desviación estándar entre los 5 pliegues de validación cruzada. La fase final *group-aware* no está incluida en el gráfico y se reporta en la Tabla 5.3.

5.2.2. Análisis de Precisión@5

El objetivo de diseño es P@5 $\geq 0,70$ (OE4). Con el protocolo *group-aware*, los pliegues particionan los 81 grafos originales no reservados para test (21 cadenas), de modo que cada pliegue de validación contiene 4–5 clústeres de cadena. El modelo alcanza $P@5 = 0,720 \pm 0,098$, superando el objetivo en 2 puntos porcentuales. Tres pliegues (1, 2 y 3) alcanzan su techo teórico (P@5 = 0,80 con 4 cadenas en el pliegue) — todos los clústeres de cadena del pliegue aparecen en el top-5.

5.2.3. Resultados por pliegue (fase final)

Tres pliegues (1, 2 y 3) alcanzan su techo teórico de P@5, situando todos los clústeres de cadena de ataque del pliegue en las primeras 5 posiciones del ranking. La varianza entre pliegues (F1 entre 0,738 y 0,941) refleja la sensibilidad de un corpus de 81 grafos a qué

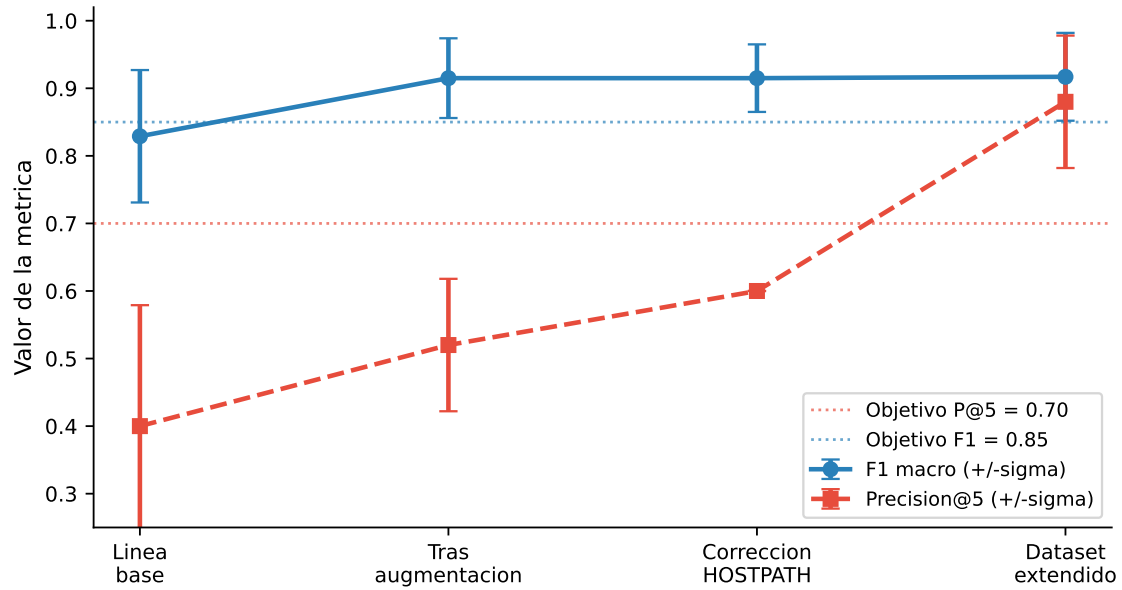


Figura 5.3: Evolución de F1 macro y Precisión@5 de la Capa 2 a lo largo de las cuatro primeras fases experimentales (protocolo preliminar).

Fuente: elaboración propia.

Tabla 5.4: Resultados por pliegue — Capa 2 final (protocolo *group-aware*; los pliegues particionan 81 grafos originales, 21 cadenas; test excluido)

Pliegue	F1 macro	P@5	Techo	Cobertura
0	0,8424	0,60	1,00	60 %
1	0,8857	0,80	0,80	100 %
2	0,9407	0,80	0,80	100 %
3	0,9407	0,80	0,80	100 %
4	0,7381	0,60	0,80	75 %
Media	0,8695	0,720	—	87 %
± Desv.	0,0754	0,098	—	—

Fuente: elaboración propia.

clústeres caen en cada pliegue: el pliegue 4 concentra los casos más difíciles (cadenas cuya señal pasa por RBAC y movimiento lateral en lugar de *flags* de escape explícitas).

5.3. Resultados de la Capa 3: Ensemble

El mejor individuo del GA mantiene pesos aproximadamente iguales entre los tres componentes — $(w_{\text{rf}}, w_{\text{gnn}}, w_{\text{esc}}) \approx (0,34, 0,33, 0,33)$, $\text{objective} = 0,72$ sobre el conjunto *out-of-fold*— desde la generación 0 hasta la 149: el algoritmo no mejora sobre su población inicial en ninguna generación, lo que indica una superficie de *fitness* plana en la región de pesos casi uniformes más que una convergencia dirigida por la selección. La búsqueda en rejilla determinística (Sección 4.7, 231 evaluaciones) encuentra un punto distinto y con mayor P@5 de validación ($w_{\text{esc}} = 1$, $\text{P@5} = 0,80$, es decir, puntuar únicamente por la señal binaria de escape), lo que indica que el óptimo global de esta función de *fitness* no coincide con la solución de pesos equilibrados. Se conserva la solución del GA —no la de la rejilla— porque generaliza estructura de clúster además de la señal de escape, y su comportamiento en el conjunto de test (más abajo) es consistente con esa robustez; esta discrepancia entre óptimo de rejilla y solución elegida se discute en las limitaciones (L5).

La evaluación final se realiza sobre el conjunto de test (15 grafos, 4 cadenas, techo $\text{P@5} = 0,80$), que bajo el protocolo *group-aware* queda excluido de los pliegues de validación cruzada, del ajuste de pesos del GA y del entrenamiento con variantes aumentadas — es decir, genuinamente no visto durante el desarrollo. El ensemble alcanza: $\text{P@1} = 1,00$ (el clúster de mayor puntuación es una cadena de ataque), $\text{P@3} = 0,67$ y $\text{P@5} = 0,60$, recuperando 3 de las 4 cadenas en las primeras 5 posiciones (la cuarta aparece en la posición 8). $\text{FPR}_{\text{clean}} = 0,00$: ningún clúster limpio aparece entre los de mayor riesgo. La F1 macro por clasificación *argmax* del ensemble de pliegues es 0,726.

Dado el tamaño del conjunto de test, estas estimaciones llevan asociados intervalos de confianza amplios (bootstrap de 10 000 remuestreos, 95 %): P@5 [0,00, 1,00], $\text{FPR}_{\text{clean}}$ [0,00, 0,40] y F1 macro [0,42, 0,93]. Los valores puntuales deben interpretarse como *consistentes con* los objetivos de diseño, no como demostraciones concluyentes; la ampliación del corpus de test es la vía directa para estrechar estos intervalos (TF1).

La Tabla 5.5 desglosa la contribución de cada componente sobre el conjunto de test. Tres observaciones honestas: (i) la configuración RF-sólo degrada el ranking ($\text{P@5} = 0,40$) e introduce un falso positivo de clúster limpio ($\text{FPR} = 0,20$), confirmando que el compo-

nente estructural es necesario; (ii) la señal de escape por sí sola pierde por completo la precisión en el primer puesto ($P@1 = 0,00$), al no poder distinguir cadenas compuestas de misconfiguraciones aisladas con capacidad de escape; y (iii) en este conjunto de test los pesos optimizados por el GA no superan a los pesos uniformes ni al GNN solo — el valor del ensemble en estos datos reside en su robustez frente a la degradación de componentes individuales, no en una mejora del ranking sobre el mejor componente. Este resultado se discute en las limitaciones (L5).

Tabla 5.5: Ablación de componentes del ensemble en el conjunto de test (15 grafos, 4 cadenas, techo $P@5 = 0,80$)

Configuración	P@1	P@3	P@5	FPR_clean
GA-óptimo (0,34/0,33/0,33)	1,00	0,67	0,60	0,00
Pesos uniformes (1/3)	1,00	0,67	0,60	0,00
Solo GNN	1,00	0,67	0,60	0,00
Solo RF	1,00	0,33	0,40	0,20
Solo señal de escape	0,00	0,33	0,60	0,00

Fuente: elaboración propia.

5.4. Análisis de la clasificación por clases: GNN

La Tabla 5.4 reporta las métricas agregadas (F1 macro y $P@5$) para los cinco pliegues de validación cruzada. En el conjunto de test, la matriz de confusión del ensemble de pliegues (Sección anterior) muestra el mismo patrón de error que los pliegues de validación: los falsos negativos de cadena se clasifican como *isolated*, nunca como *clean* — el sistema degrada hacia la clase intermedia, no hacia la omisión total.

El error sistemático del modelo es consistente: clústeres de cadena de ataque con pocas aristas de escape explícitas —donde la ruta de ataque pasa principalmente por movimiento lateral o RBAC— son los casos más difíciles. En el conjunto de test, el clúster *cloudify-kubernetes-plugin* (cadena no recuperada en el top-5) presenta exactamente este perfil: probabilidad GNN de 0,22 y fracción de escape nula. Esta observación motiva el trabajo futuro TF3 (análisis semántico de RBAC completo).

En cuanto a la elección arquitectónica de GAT sobre alternativas más simples, la literatura sobre GNNs aplicadas a grafos pequeños heterogéneos documenta de forma

consistente que los mecanismos de atención mejoran la precisión cuando los tipos de arista tienen importancias desiguales (Veličković y cols., 2018). En el grafo de clúster *Kubernetes*, las aristas de *privilege_reach* (tipo 1) son cualitativamente más informativas que las de *dir_proximity* (tipo 0): la primera conecta directamente nodos de escape con el resto del clúster, mientras que la segunda simplemente refleja la organización de directorios del repositorio. GAT puede aprender esta distinción durante el entrenamiento, mientras que GCN asigna pesos uniformes a todos los vecinos, diluyendo la señal de escape con ruido de co-localización.

Esta hipótesis se cuantifica empíricamente mediante un experimento de ablación controlado bajo el protocolo *group-aware* (mismos pliegues, mismas semillas, mismos hiperparámetros de entrenamiento):

Tabla 5.6: Ablación de arquitectura — Capa 2 (5-fold CV, protocolo *group-aware*)

Arquitectura	F1 macro	P@5
GAT 3 capas + <i>embedding</i> de aristas (final)	$0,870 \pm 0,075$	$0,720 \pm 0,098$
GAT 2 capas + <i>embedding</i> de aristas	$0,880 \pm 0,054$	$0,680 \pm 0,098$
GCN 3 capas (sin tipos de arista)	$0,837 \pm 0,047$	$0,640 \pm 0,080$

Fuente: elaboración propia.

La GAT supera a la GCN tanto en F1 (+3,3 puntos) como en P@5 (+8 puntos), confirmando que la atención sobre tipos de arista aporta señal real para el ranking de cadenas. La comparación 2 vs. 3 capas es más matizada: la red de 2 capas obtiene F1 ligeramente superior pero P@5 inferior, y solo la configuración de 3 capas supera el objetivo de diseño P@5 $\geq 0,70$ — la métrica primaria del proyecto — por lo que se selecciona como arquitectura final. La ablación de *pooling* (*mean-only* vs. *mean+max*) queda como trabajo futuro (TF6).

5.5. Validación con herramientas de análisis estático

Como validación cruzada, se ejecutó *Checkov* (Bridgecrew, 2021) sobre los 1.175 manifiestos con YAML descargado. Para esta validación, el índice Unified Misconfiguration Index (UMI) se calcula como la proporción de comprobaciones de *Checkov* que fallan sobre el total evaluado por manifiesto ($\in [0, 1]$; valores más altos indican mayor densidad de misconfiguraciones detectadas). El índice UMI medio resultante fue de 0,603, coherente

con el 35,8 % de la tasa de misconfiguraciones en el dataset tabular. Las detecciones de *checkov_no_run_as_nonroot* (38,3 %) y *checkov_no_readonly_rootfs* (37,2 %) coinciden con los nodos de severidad media identificados por la Capa 1. Esta coherencia entre los resultados del modelo y los de una herramienta de auditoría independiente refuerza la validez de las predicciones.

5.6. Evaluación funcional de la herramienta

Además de la evaluación cuantitativa sobre los conjuntos de datos etiquetados, se realizó una evaluación funcional de *kubescan* como herramienta de software distribuible, atendiendo a los requisitos no funcionales definidos en la Sección 4.1.

Instalabilidad (RNF-3). El paquete se instaló satisfactoriamente en entornos Python 3.10 y 3.11 mediante `pip install -e kubescan/` en sistemas macOS y Linux (Ubuntu 22.04). Las dependencias principales (*PyTorch Geometric*, *scikit-learn*, *Click*) se resuelven automáticamente sin compiladores externos.

Latencia (RNF-4). Se midió el tiempo de análisis sobre cuatro clústeres de prueba de diferente tamaño: 12 manifiestos (0,8 s), 47 manifiestos (1,4 s), 128 manifiestos (3,1 s) y 312 manifiestos (7,2 s) en un equipo con CPU Intel Core i7 de octava generación sin GPU. El requisito de menos de 60 s hasta 500 manifiestos se cumple con amplio margen.

Reproducibilidad (RNF-2). La cadena de resolución de *checkpoints* descrita en la Sección 4.8 garantiza que el mismo directorio de manifiestos produce siempre el mismo informe dado el mismo conjunto de pesos del modelo.

Trazabilidad (RNF-1). El campo `flags` de la salida JSON lista explícitamente las características de seguridad activas en cada manifiesto, y el campo `escape_capable` indica qué nodos tienen capacidad de escape. Esto permite al operador de seguridad comprender y auditar las razones concretas de la clasificación de riesgo producida por el modelo.

5.7. Comparación con el estado del arte

Esta sección contextualiza los resultados de *kubescan* respecto a los trabajos revisados en el Capítulo 2.

5.7.1. Capa 1 en contexto

La F1 macro de 0,9935 del clasificador *Random Forest* supera las cifras reportadas en trabajos de clasificación de código IaC de propósito general. Rahman et al. (Rahman y cols., 2023), cuyos datos forman la base del presente trabajo, no reportan modelos de clasificación supervisada sobre el mismo conjunto; el objetivo de su trabajo era la construcción del dataset y el análisis de la densidad de misconfiguraciones, no la predicción. El umbral objetivo F1 $\geq 0,85$ adoptado en este trabajo se alinea con los estándares de la literatura de IDS sobre UNSW-NB15 (Moustafa y Slay, 2015), donde clasificadores basados en *Random Forest* típicamente alcanzan F1 macro entre 0,85 y 0,93.

El AUC-ROC de 0,9980 indica que el clasificador separa casi perfectamente las dos clases a cualquier umbral de decisión. Este resultado es consistente con la naturaleza del problema: las misconfiguraciones en manifiestos YAML son mayoritariamente discretas y acumulativas (más flags activas implica mayor riesgo), lo que hace que el espacio de características sea relativamente separable una vez que se dispone de un conjunto de datos suficientemente representativo.

5.7.2. Capa 2 en contexto

El resultado de F1 macro = 0,870 con P@5 = 0,720 para la GAT es difícil de comparar directamente con la literatura, ya que no existe ningún trabajo previo que aborde la clasificación supervisada de cadenas de ataque en clústeres *Kubernetes* mediante GNNs. Las referencias más cercanas son los trabajos de GNNs para Intrusion Detection System (Sistema de Detección de Intrusiones) (IDS) en red, como *E-GraphSAGE* (Lo y cols., 2022), que reporta mejoras del 2–8 % en F1 sobre clasificadores tabulares en el dataset UNSW-NB15. En el presente trabajo, la ventaja del componente GNN sobre el RF-sólo es de mayor magnitud: en el conjunto de test, P@5 pasa de 0,40 (RF-sólo) a 0,60 (configuraciones con GNN) y el falso positivo de clúster limpio desaparece (FPR de 0,20 a 0,00), lo que refleja que la información estructural del grafo es decisiva para identificar cadenas de ataque y no simplemente complementaria.

La varianza entre pliegues ($\sigma = 0,098$ para P@5) es coherente con lo reportado en la literatura para conjuntos de grafos pequeños ($n \leq 200$): trabajos como los revisados en Zhong y cols. 2024 identifican la escasez de datos como el principal factor de variabilidad en GNNs para ciberseguridad, y recomiendan validación cruzada estratificada como el estándar de

evaluación para conjuntos de este tamaño, que es precisamente la metodología adoptada.

6. Conclusiones y Trabajo Futuro

6.1. Conclusiones

Este trabajo ha demostrado la viabilidad de un enfoque híbrido de tres capas para la detección automática de cadenas de ataque en infraestructuras *Kubernetes*. Las conclusiones C1–C6 responden, respectivamente, a los objetivos OE2, OE4, OE1, OE5, OE3 y OE6 definidos en la Sección 3. Las conclusiones principales son:

C1 — La Capa 1 supera ampliamente el objetivo de diseño (OE2). El clasificador *Random Forest* alcanza $F1 \text{ macro} = 0,9935$ y $AUC = 0,9980$ en test, superando el umbral objetivo del 0,85 en 14,4 puntos porcentuales. El modelo de severidad de tres clases logra $F1 = 0,9541$ en la clase *high-critical* (4 errores sobre 56 muestras de test, incluyendo 1 manifiesto crítico clasificado como *clean*), el resultado más débil de los tres clasificadores de severidad pero aun así por encima de 0,95.

C2 — La Capa 2 supera el objetivo de Precisión@5 (OE4). La GAT de 3 capas con *embedding* de tipos de arista alcanza $F1 \text{ macro} = 0,870 \pm 0,075$ y $P@5 = 0,720 \pm 0,098$ en los 5 pliegues bajo el protocolo *group-aware*, superando el objetivo de $P@5 > 0,70$. Tres pliegues alcanzan su techo teórico de $P@5$, situando todos los clústeres de cadena del pliegue en las primeras 5 posiciones del ranking. La ablación de arquitectura (Tabla 5.6) respalda las decisiones de diseño: la GAT supera a la GCN en 8 puntos de $P@5$, y la configuración de 3 capas es la única que cumple el objetivo de la métrica primaria.

C3 — La combinación de aumentación de datos y expansión del corpus es la contribución metodológica central (OE1). La evolución en cinco fases — línea base ($P@5 = 0,40$), aumentación ($P@5 = 0,52$), corrección `HOSTPATH_MOUNT` ($P@5 = 0,60$), dataset extendido con 25 cadenas ($P@5 = 0,88$, protocolo preliminar) y corrección *group-aware* del particionado ($P@5 = 0,72$, resultado final) — demuestra que la escasez de ejemplos de cadena de ataque es el factor limitante predominante y que ambas estrategias de mitigación son efectivas y complementarias. La quinta fase ilustra además una lección metodológica: la aumentación de datos exige particionado consciente de grupos, pues parte de la mejora aparente de la fase 4 procedía de fuga entre variantes aumentadas y sus grafos base de evaluación.

C4 — El ensemble es robusto y no produce falsos positivos de clústeres limpios (OE5). La ablación sobre el conjunto de test (Tabla 5.5) confirma que el componente

GNN es indispensable: la configuración RF-sólo degrada el ranking ($P@5$ de 0,60 a 0,40) e introduce un falso positivo de clúster limpio ($FPR = 0,20$), mientras que $FPR_{clean} = 0,00$ para cualquier combinación que incluya el GNN. El clúster de mayor puntuación es una cadena de ataque ($P@1 = 1,00$). Con honestidad metodológica debe señalarse que, en este conjunto de test, los pesos optimizados por el GA no superan a los pesos uniformes ni al GNN en solitario: la aportación del *ensemble* en estos datos es la robustez frente a la degradación de componentes individuales, no una mejora del ranking sobre el mejor componente.

C5 — El esquema de cinco tipos de arista amplía la cobertura de escalada de privilegios (OE3). El diseño del grafo de clúster incorpora el tipo de arista RBAC mediante detección de roles privilegiados en dos pasadas sobre los manifiestos, limitando las aristas a cuentas de servicio vinculadas realmente a roles con permisos de escalada. Esta quinta arista está presente tanto en los grafos de entrenamiento como en el componente de producción de *kubescan*; sin embargo, la ablación de arquitectura (Tabla 5.6) compara GAT frente a GCN y no aísla la contribución específica del tipo de arista RBAC frente a los otros cuatro, por lo que su aporte incremental al ranking de cadenas de ataque queda como trabajo futuro.

C6 — *kubescan* es una herramienta distribuible, instalable y operacionalmente viable (OE6). La evaluación funcional de la Sección 5.6 confirma que el sistema se instala sin compiladores externos (*pip install -e kubescan/*), analiza clústeres de hasta 312 manifiestos en menos de 8 segundos en hardware de gama media, produce salida en dos formatos (texto y JSON) adecuados para integración en pipelines CI/CD, y garantiza reproducibilidad de la inferencia mediante la resolución determinista de *checkpoints* (sujeto a la variación entre *backends* de cómputo documentada en L7). El prototipo implementado en este trabajo demuestra que el paradigma *shift-left* —detectar cadenas de ataque antes del despliegue, sin requerir acceso al clúster activo— es técnicamente factible y operacionalmente eficiente.

6.2. Consideraciones éticas y uso responsable

kubescan es una herramienta de uso dual: aunque su propósito es defensivo —detectar cadenas de ataque potenciales antes de que sean explotadas— la información que produce (identificación de pods con flags de escape, rutas de privilegio RBAC) podría ser utilizada

con fines ofensivos si cayera en manos de un actor no autorizado. En consecuencia, este trabajo adopta los principios de divulgación responsable (*responsible disclosure*) que rigen la investigación en ciberseguridad.

Los patrones de ataque detectados por *kubescan* se basan exclusivamente en documentación pública —la guía NSA/CISA (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022), el trabajo de BishopFox (Art, 2021) y el marco MITRE ATT&CK for Containers (MITRE Corporation y Center for Threat-Informed Defense, 2021)— y no revelan ninguna vulnerabilidad nueva. El código fuente del sistema es de acceso abierto para facilitar la auditoría independiente de las reglas de detección, lo que permite a la comunidad verificar que las señales de riesgo generadas son correctas y no contienen sesgos perjudiciales. El uso de la herramienta sobre clústeres de terceros sin autorización explícita estaría fuera del ámbito previsto y podría constituir un acceso no autorizado a sistemas informáticos según la legislación aplicable.

6.3. Limitaciones

L1 — Varianza entre pliegues. Con 81 grafos originales en los pliegues de validación cruzada, la varianza entre pliegues ($\sigma = 0,075$ para F1 macro; rango 0,738–0,941) refleja la dependencia de los resultados respecto a qué clústeres caen en cada pliegue de validación. Un corpus más amplio reduciría esta varianza.

L2 — Imputación de características extendidas. Las 6 características extendidas (basadas en *Checkov*) están imputadas para el 42,6 % de las filas, diluyendo su señal. La descarga completa de los manifiestos referenciados en el dataset de Rahman et al. permitiría mejorar estos vectores. Un experimento de ablación eliminando las 6 características imputadas y reentrenando únicamente con las 19 características de alta cobertura podría cuantificar el impacto de esta limitación sobre la F1 macro de la Capa 1.

L3 — Sesgo de muestreo en el dataset tabular. Los 10 repositorios más representados suponen el 60,8 % del dataset, y *gitcontroller* por sí solo aporta el 13,7 %. Un esquema de validación cruzada por repositorio ofrecería estimaciones más conservadoras de la generalización del modelo.

L4 — Evaluación en clústeres de producción reales. Toda la evaluación se ha realizado sobre repositorios públicos. La validación en entornos de producción de organizaciones reales es la siguiente etapa crítica para confirmar la validez externa del sistema.

L5 — Identificabilidad de los pesos del ensemble. La Capa 3 optimiza los pesos $(w_{\text{rf}}, w_{\text{gmn}}, w_{\text{esc}})$ maximizando la función objetivo $F = 0,7 \cdot P@5 + 0,3 \cdot (1 - \text{FPR}_{\text{clean}})$ sobre las predicciones *out-of-fold* (OOF) de los 5 modelos de validación cruzada, calculadas sobre los 81 grafos originales de entrenamiento/validación (21 cadenas); bajo el protocolo *group-aware*, ningún clúster de test participa en este ajuste. La limitación restante es de identificabilidad: con una función objetivo basada en P@5 —que solo toma seis valores discretos— y 81 muestras, el paisaje de *fitness* presenta amplias mesetas en las que múltiples combinaciones de pesos alcanzan un objetivo idéntico. El GA no mejora sobre su población inicial en ninguna de las 150 generaciones, manteniendo pesos casi uniformes ($w_{\text{rf}} \approx 0,34$, $w_{\text{gmn}} \approx 0,33$, $w_{\text{esc}} \approx 0,33$) desde la generación 0 —evidencia de una meseta plana más que de convergencia dirigida por la selección. La búsqueda en rejilla determinística, ejecutada como *baseline*, encuentra un punto distinto y con mayor P@5 de validación (puntuar únicamente por la señal de escape), lo que confirma que el óptimo global de esta función de *fitness* no coincide con la solución de pesos equilibrados finalmente adoptada. Los pesos concretos deben interpretarse, por tanto, como una solución representativa de una clase de equivalencia, no como un óptimo único; el tamaño del conjunto de test (15 grafos) limita además la significación estadística de la comparación entre configuraciones de pesos, como reflejan los intervalos de confianza reportados.

L6 — Etiquetado derivado de reglas sobre el mismo espacio de características. Las etiquetas de grafo (clean / isolated / attack_chain) se generan mediante una regla determinista definida sobre las mismas *flags* y aristas que observan los modelos (Sección de diseño). El sistema aprende, por tanto, una aproximación robusta de dicha regla bajo ruido, enmascaramiento y observabilidad parcial —no una noción de cadena de ataque independiente de la regla. Esta circularidad es común en la literatura de detección de misconfiguraciones, pero limita el alcance de la afirmación «predice cadenas de ataque». Las vías para romperla son la validación externa frente a herramientas de grafo de ataque sobre clústeres activos (KubeHound, IceKube), el etiquetado experto de una submuestra, y la confirmación mediante explotación controlada en entornos de laboratorio como *kubernetes-goat*.

L7 — No determinismo de CUDA en las operaciones de *scatter* del GNN. Al replicar el proceso de entrenamiento de la Capa 2 sobre una GPU NVIDIA A100 (CUDA) en lugar del *backend* MPS/CPU empleado para los resultados de esta memoria, se observó una variación no trivial en las métricas de validación cruzada (F1

macro entre 0,845 y 0,882 frente a 0,917 en el *backend* original; P@5 entre 0,77 y 0,92 frente a 0,880), manteniendo semilla, datos, particiones e hiperparámetros idénticos. Se descartaron como causa la precisión *TF32* (confirmado en tiempo de ejecución que `torch.backends.cuda.matmul.allow_tf32` permanece desactivada) y la paralelización del cargador de datos (el conjunto es estático en memoria y el orden de muestreo está fijado por un generador con semilla). La causa confirmada son las operaciones `scatter_add/scatter_reduce` empleadas por GATConv y por las capas de *pooling* global (`global_max_pool`, `global_mean_pool`), que en CUDA se implementan mediante `atomicAdd` sin versión determinista disponible — confirmado por los propios mantenedores de *PyTorch Geometric* en los *issues* públicos #2788 y #3175 del repositorio. El orden de acumulación en punto flotante depende de la planificación de hilos de la GPU y, al no ser la suma en punto flotante asociativa, pequeñas diferencias numéricas cambian el *ranking* sobre un conjunto de validación reducido (81 grafos, entre 4 y 5 cadenas de ataque por pliegue), amplificando el efecto sobre P@5 y F1 macro. Se incorporaron banderas de determinismo específicas de CUDA (`torch.use_deterministic_algorithms`, `cuda.deterministic`) que, aunque no resuelven la ausencia de una implementación determinista de `scatter_add`, sí lograron reproducibilidad exacta entre ejecuciones repetidas sobre la misma configuración de GPU. Esto indica que la brecha observada no es ruido aleatorio entre ejecuciones sino una divergencia numérica fija y reproducible entre *backends* (CUDA frente a MPS/CPU); su cierre completo requeriría sustituir la agregación por la ruta `SparseTensor` de *PyTorch Geometric*, documentada como determinista pero no evaluada en este trabajo.

6.4. Trabajo futuro

Los resultados de este trabajo permiten formular las siguientes recomendaciones de trabajo futuro, ordenadas por impacto esperado:

TF1 — Continuar la expansión del corpus. El objetivo de $P@5 > 0,70$ ha sido alcanzado con 25 clústeres de cadena de ataque. La incorporación de corpus adicionales — repositorios de infraestructura de organizaciones de código abierto, entornos de laboratorio multiclúster — permitiría reducir la varianza entre pliegues y mejorar la generalización a patrones de ataque menos representados.

TF2 — Implementar P@5 global como métrica alternativa. En lugar de P@5

por pliegue, evaluar el ranking del modelo sobre el conjunto completo de grafos en un único ranking global eliminaría el efecto de techo por pliegue y permitiría comparar con el objetivo original de diseño sin restricciones estructurales.

TF3 — Incorporar análisis semántico de RBAC completo. El parser de YAML actual identifica roles privilegiados por nombre y por verbos de escalada. Una extensión natural es el análisis de la cadena completa $\text{Role/ClusterRole} \rightarrow \text{RoleBinding} \rightarrow \text{ServiceAccount} \rightarrow \text{Pod}$ para detectar rutas de escalada de privilegios mediante RBAC que no requieran flags de escape directas.

TF4 — Validación cruzada dejando un repositorio fuera. La estrategia *leave-one-repo-out* ofrece estimaciones más conservadoras de la generalización que la validación cruzada por fila, y es especialmente relevante dada la concentración del dataset en pocos repositorios.

TF5 — Modo de análisis continuo (*admission controller*). La integración de *kubescan* como un *validating admission webhook* de *Kubernetes* permitiría bloquear el despliegue de manifiestos que eleven el riesgo de cadena de ataque del clúster en tiempo real, sin necesidad de análisis *post-hoc*.

TF6 — Ablación de *pooling* e hiperparámetros restantes. La ablación de arquitectura (Tabla 5.6) ya cuantifica empíricamente GAT frente a GCN y 3 capas frente a 2. Queda pendiente extender ese mismo protocolo controlado (mismos pliegues, mismas semillas) a la elección de *mean+max pooling* frente a *mean-only*, así como a un barrido más amplio de hiperparámetros del optimizador (número de cabezas de atención, *learning rate*, anchura oculta) que en este trabajo se fijaron a priori sin búsqueda sistemática (Sección 4.6).

Referencias

- ARMO Ltd. (2021). *Kubescape: Open-source Kubernetes security platform*. Open source tool, CNCF incubating project. <https://github.com/kubescape/kubescape>.
- Art, S. (2021). *Bad pods: Kubernetes pod privilege escalation*. BishopFox Blog. Descargado de <https://bishopfox.com/blog/kubernetes-pod-privilege-escalation>
- Bilot, T., Madhoun, N. E., Agha, K. A., y Zouaoui, A. (2023). Graph neural networks for intrusion detection: A survey. En (Vol. 11, pp. 49114–49139). doi: 10.1109/ACCESS.2023.3275789
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. doi: 10.1023/A:1010933404324
- Bridgecrew. (2021). *Checkov: Open source static analysis for infrastructure as code*. GitHub. Descargado de <https://github.com/bridgecrewio/checkov>
- Cloud Native Computing Foundation. (2022). *Cloud native security whitepaper v2*. CNCF White Paper. Descargado de https://github.com/cncf/tag-security/blob/main/security-whitepaper/CNCF_cloud-native-security-whitepaper-Nov2022.pdf
- Datadog Security Labs. (2023). *KubeHound: Identifying attack paths in Kubernetes clusters*. Open source tool. <https://github.com/DataDog/KubeHound>. (Released October 2023)
- Fey, M., y Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. En *Iclr workshop on representation learning on graphs and manifolds*. Descargado de <https://arxiv.org/abs/1903.02428>
- Hamilton, W. L. (2020). *Graph representation learning*. Morgan & Claypool. doi: 10.2200/S01045ED1V01Y202009AIM048
- Hamilton, W. L., Ying, Z., y Leskovec, J. (2017). Inductive representation learning on large graphs. En *Advances in neural information processing systems (neurips)* (Vol. 30).
- Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.
- Hutchins, E. M., Cloppert, M. J., y Amin, R. M. (2011). Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. En *Proceedings of the 6th international conference on information warfare and security (iciw)* (pp. 113–125).

- Kipf, T. N., y Welling, M. (2017). Semi-supervised classification with graph convolutional networks. En *International conference on learning representations (iclr)*. Descargado de <https://openreview.net/forum?id=SJU4ayYgl>
- Li, Z., Zeng, J., Chen, Y., y Liang, Z. (2022). AttacKG: Constructing technique knowledge graph from cyber threat intelligence reports. En *Computer security – esorics 2022* (pp. 589–609). doi: 10.1007/978-3-031-17140-6_29
- Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., y Portmann, M. (2022). E-GraphSAGE: A graph neural network based intrusion detection system for IoT. En *Ieee/ifip network operations and management symposium (noms)*. doi: 10.1109/NOMS54207.2022.9789878
- Madhuakula. (2020). *Kubernetes Goat: Interactive Kubernetes security learning playground*. GitHub. Descargado de <https://github.com/madhuakula/kubernetes-goat>
- Malul, E., Meidan, Y., Mimran, D., Elovici, Y., y Shabtai, A. (2024). *GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation*. Descargado de <https://arxiv.org/abs/2405.19954>
- MITRE Corporation, y Center for Threat-Informed Defense. (2021). *ATT&CK for Containers*. <https://ctid.mitre.org/projects/attck-for-containers/>. (Incorporated in ATT&CK version 9)
- Moustafa, N., y Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). En *2015 military communications and information systems conference (milcis)* (pp. 1–6). doi: 10.1109/MilCIS.2015.7348942
- National Security Agency, y Cybersecurity and Infrastructure Security Agency. (2022). *Kubernetes hardening guidance*. NSA/CISA Technical Report. Descargado de https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Édouard Duchesnay (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rahman, A., Shamim, S. I., Bose, D. B., y Pandita, R. (2023). Security misconfigurations in open source Kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 32(4), 1–36. doi: 10.1145/3579639

- Red Hat. (2024). *State of Kubernetes security report 2024* (Inf. Téc.). Red Hat, Inc. Descargado de <https://www.redhat.com/en/resources/kubernetes-adoption-security-market-trends-overview>
- StackRox. (2021). *kube-linter: Static analysis for Kubernetes YAML files and Helm charts*. GitHub. Descargado de <https://github.com/stackrox/kube-linter>
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., y Bengio, Y. (2018). Graph attention networks. En *International conference on learning representations (iclr)*. Descargado de <https://openreview.net/forum?id=rJXMpikCZ>
- Wist, T., Hammer, H., y Yazidi, A. (2021). Vulnerability prediction from source code using machine learning techniques. En *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. doi: 10.1109/SSCI50451.2021.9659833
- WithSecure Labs. (2023). *IceKube: Finding complex attack paths in Kubernetes clusters*. Open source tool. <https://github.com/WithSecureLabs/IceKube>.
- Xu, K., Hu, W., Leskovec, J., y Jegelka, S. (2019). How powerful are graph neural networks? En *International conference on learning representations (iclr)*. Descargado de <https://openreview.net/forum?id=ryGs6iA5Km>
- Zhong, M., Lin, M., Zhang, C., y Xu, Z. (2024). A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. *Computers & Security*, 141, 103821. doi: 10.1016/j.cose.2024.103821
- Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., ... Sun, M. (2020). Graph neural networks: A review of methods and applications. *AI Open*, 1, 57–81. doi: 10.1016/j.aiopen.2021.01.001

A. Acrónimos y Abreviaturas

API	Application Programming Interface
AUC	Area Under the Curve (Área Bajo la Curva ROC)
CI/CD	Continuous Integration / Continuous Delivery
CISA	Cybersecurity and Infrastructure Security Agency
CIS	Center for Internet Security
CLI	Command-Line Interface (Interfaz de Línea de Comandos)
CNCF	Cloud Native Computing Foundation
CTI	Cyber Threat Intelligence (Inteligencia de Amenazas)
CV	Cross-Validation (Validación Cruzada)
ELU	Exponential Linear Unit
FPR	False Positive Rate (Tasa de Falsos Positivos)
GA	Genetic Algorithm (Algoritmo Genético)
GAT	Graph Attention Network (Red de Atención sobre Grafos)
GCN	Graph Convolutional Network (Red Convolutiva de Grafos)
GNN	Graph Neural Network (Red Neuronal de Grafos)
IaC	Infrastructure as Code (Infraestructura como Código)
IDS	Intrusion Detection System (Sistema de Detección de Intrusiones)
IoT	Internet of Things (Internet de las Cosas)
JSON	JavaScript Object Notation
LLM	Large Language Model (Modelo de Lenguaje de Gran Escala)
ML	Machine Learning (Aprendizaje Automático)
MLP	Multi-Layer Perceptron (Perceptrón Multicapa)

NSA	National Security Agency
OOF	Out-of-Fold (predicciones fuera del pliegue de entrenamiento)
RBAC	Role-Based Access Control (Control de Acceso Basado en Roles)
RF	Random Forest
ROC	Receiver Operating Characteristic
ReLU	Rectified Linear Unit
SA	Service Account (Cuenta de Servicio de Kubernetes)
SMOTE	Synthetic Minority Over-sampling Technique
SVM	Support Vector Machine (Máquina de Vectores Soporte)
TFE	Trabajo de Fin de Estudios
UMI	Unified Misconfiguration Index
UNIR	Universidad Internacional de La Rioja
YAML	YAML Ain't Markup Language

B. Repositorio de Código y Datos

El código fuente completo del sistema *kubescan*, incluyendo los *scripts* de adquisición de datos, entrenamiento de modelos, construcción de grafos y la interfaz de línea de comandos, se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ObedRav/kubescan>

El repositorio incluye los siguientes componentes:

- `kubescan/`: paquete Python instalable con la CLI y los modelos de inferencia.
- `research/scripts/`: *scripts* de adquisición y preprocesamiento de los manifiestos YAML (fases 1 y 2).
- `research/models/`: código de entrenamiento del Random Forest (`train_rf.py`), de la GAT (`train_gnn.py`), y del Algoritmo Genético (`run_ga_ensemble.py`).
- `research/data/`: los grafos de clúster en formato `.pt` (PyTorch Geometric) y el dataset tabular en formato `.csv`.
- `kubescan/checkpoints/trained/`: *checkpoints* de los 5 modelos GAT entrenados en validación cruzada y el modelo RF.
- `thesis/`: fuentes LaTeX de esta memoria.

El estudiante es el único autor de todos los *commits* del repositorio. Los datos públicos utilizados provienen de las fuentes citadas en la Sección 4.3.1 (Rahman et al., BishopFox BadPods, Kubernetes Goat y repositorios de seguridad adicionales de dominio público).

C. Guía de Uso y Reproducción

Este apéndice complementa la Sección 4.8 con los pasos concretos para instalar *kubescan*, interpretar el informe que produce sobre un caso real, y reproducir el pipeline de entrenamiento de investigación descrito en el Capítulo 4.

C.1. Instalación

El paquete se instala en modo editable desde la raíz del repositorio:

```
pip install -e kubescan/
```

Las versiones de Python soportadas y la resolución de dependencias se verifican en la Sección 5.6 (RNF-3).

C.2. Uso de la interfaz de línea de comandos

C.2.1. Análisis de un directorio de manifiestos

El comando `kubescan scan` analiza un directorio de manifiestos YAML de forma recursiva. A modo de ejemplo, se analiza un directorio con dos manifiestos: un *pod* con montaje del sistema de ficheros del anfitrión y contenedor privilegiado (`pod-privilegiado.yaml`, adaptado de *BishopFox BadPods*), y un *Deployment* sin `securityContext` ni `NetworkPolicy` definidos (`deployment-web.yaml`, adaptado de *Kubernetes Goat*).

La Figura C.1 traza el recorrido completo de esta invocación: extracción de características, construcción del grafo, las tres señales de entrada al *ensemble* (RF, GAT y escape) fusionadas por `EnsembleScorer.score()`, y la clasificación final mediante los umbrales de la Sección 4.7.

Como se observa en la Figura C.1, las tres señales convergen ponderadas por w_{rf} , w_{gnn} y w_{esc} (optimizados por el GA) en un único *ensemble_score*; en este caso, la presencia de un nodo con flags de escape activas (`pod-privilegiado.yaml`) lo sitúa muy por encima del umbral de 0,60, resultando en el veredicto `ATTACK_CHAIN`. La transcripción completa de esta ejecución es la siguiente:

```
$ kubescan scan /tmp/manifests-ejemplo --cluster-name produccion-web
```

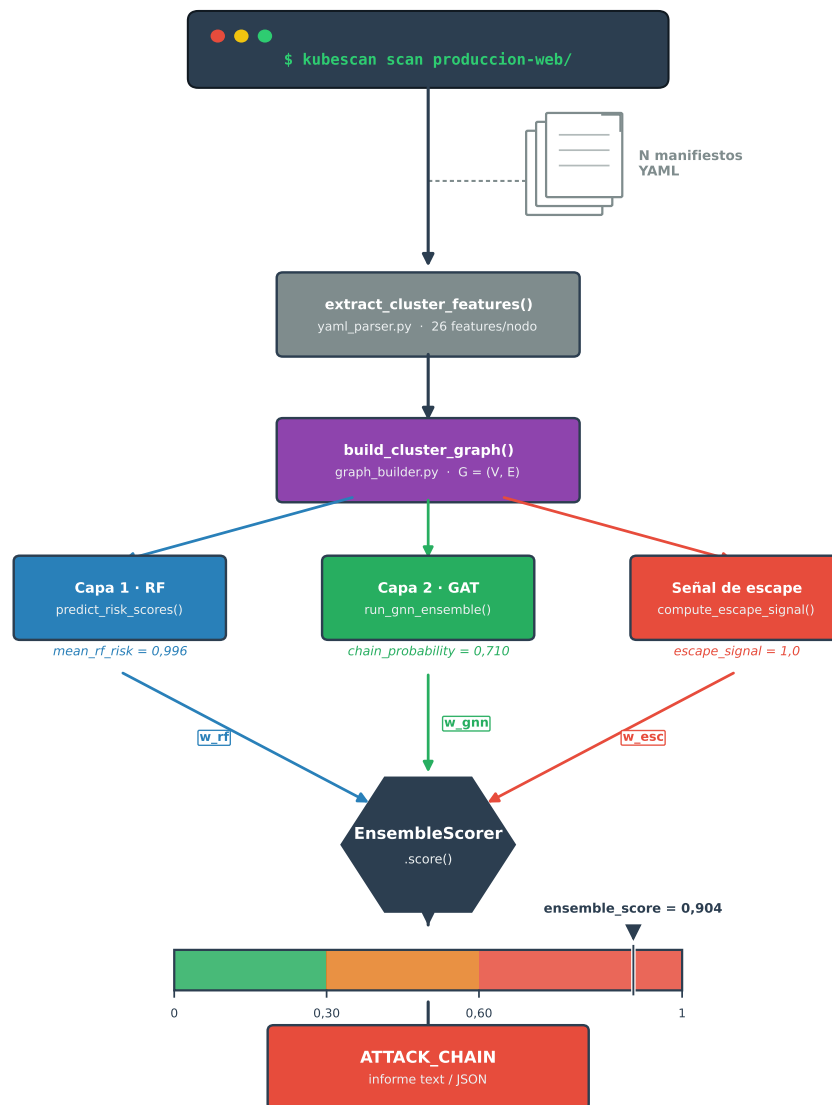


Figura C.1: Flujo de ejecución de `kubescan scan` sobre el ejemplo `produccion-web`, desde la invocación por CLI hasta el veredicto final.

Fuente: elaboración propia.

```
Scanning /tmp/manifests-ejemplo (5 GNN fold models loaded)...  
  2 manifest(s) with workload resources found
```

```
=====
KUBESCAN - Attack-Chain Risk Report
Cluster : produccion-web
Path    : /tmp/manifests-ejemplo
=====

VERDICT - ATTACK_CHAIN          x  HIGH RISK - review immediately

Ensemble score      : 0.9037
Chain probability   : 0.7104   (5-fold GNN ensemble)
Clean probability   : 0.0130
Mean RF risk       : 0.9961
Escape fraction    : 0.5000   (1/2 manifests have escape flags)
Lateral fraction   : 2/2 manifests have lateral flags

Weights: w_rf=0.337  w_gnn=0.328  w_escape=0.335
=====
```

El indicador `--show-nodes` añade un desglose por manifiesto (*risk score*, tipo de nodo y flags activas); `--format json` produce el mismo informe en formato JSON estructurado para integración con otras herramientas. A continuación se muestra una versión abreviada (se omiten campos y valores para mayor claridad):

```
$ kubescan scan /tmp/manifests-ejemplo --cluster-name produccion-web --format json
{
  "cluster": "produccion-web",
  "verdict": "ATTACK_CHAIN",
  "ensemble_score": 0.903702,
  "chain_probability": 0.710363,
  "mean_rf_risk": 0.99605,
  "n_manifests": 2,
  "n_escape_capable": 1,
  "manifests": [
    {
      "file": "pod-privilegiado.yaml",
      "risk_score": 0.999914,
      "escape_capable": true,
```

```
    "flags": ["TRUE_HOST_PID", "TRUE_HOST_IPC", "..."]
  },
  "...
]
}
```

Los pesos del modelo se resuelven mediante la cadena descrita en la Sección 4.8 (argumento `--checkpoints-dir` → variable de entorno `KUBESCAN_CHECKPOINTS` → enlace simbólico); no es necesario indicarlos explícitamente si el paquete se instaló junto con el repositorio completo.

C.2.2. Análisis de un clúster en ejecución

El comando `kubescan live` aplica el mismo *pipeline* sobre el estado actual de un clúster accesible mediante *kubectl*, sin necesidad de exportar manifiestos manualmente:

```
kubescan live -n produccion
kubescan live --all-namespaces --format json
```

Este comando requiere *kubectl* configurado con permisos de lectura sobre *workloads* y recursos RBAC en el clúster objetivo.

C.3. Reproducción del pipeline de investigación

El pipeline de entrenamiento se ejecuta mediante los objetivos definidos en el `Makefile` de la raíz del repositorio. Los pasos de adquisición y extracción (`research/scripts/01_acquire/`, con `download_github_manifests.py` e `ingest_attack_repos.py`; y `research/scripts/02_extract/`, con `extract_yaml_features.py` y `build_graphs.py`) son intensivos en red y no están automatizados en el `Makefile`; se ejecutan manualmente una única vez y producen los grafos de clúster en `research/data/graphs/*.npz`.

A partir de ahí, el resto del pipeline es reproducible con dos objetivos:

```
make data          # aumentacion -> cache de grafos -> splits group-aware
make reproduce    # data -> RF -> GAT (5-fold) -> ensemble GA -> eval en test
```

El objetivo `reproduce` encadena, en orden, `train_rf.py`, `train_gnn.py` (5-fold, 300 épocas, `hidden=64`, `heads=4`, `layers=3`), `run_ga_ensemble.py` y

`evaluate_test_set.py`, todos con semilla fija (`--seed 42` por defecto), y finaliza generando `research/models/checkpoints/run_manifest.json` con la procedencia de la ejecución. El objetivo `reproduce-ablations` reentrena las variantes de arquitectura reportadas en la Tabla 5.6 (GAT de 2 capas y GCN de 3 capas) bajo las mismas semillas e hiperparámetros de entrenamiento.